PASADHI · FULL PAPER

PASADHI: a matrix-first pool built from the machines you already own

Full paper with the complete evidence base · 46 figures

PASADHI contributors · v1.1

23 October 2026 · MIT for the open core

github.com/camponggogo/pasadhi · pasadhi.tech

Full paper · v1.1 · 23 October 2026 · PASADHI contributors · MIT for the open core

Companion to the solution paper. This volume carries the whole evidence base: every experiment, every figure, the installation procedure, the price model and the limits. Numbers are reported as measured; where a figure is a model rather than a measurement, the caption says so.

Abstract

An office of 100–200 machines already owns the capacity it rents from the cloud, but not the ability to use it as one computer. PASADHI is a matrix-first pool: one tiled kernel serves GEMM, int8 GEMM, FFT and convolution; five process layers (transport, policy, scheduler, kernel, storage) each own a bounded queue; a node plane separated from the gateway absorbs heartbeats; a content-addressed prefix cache shortens the median; and contributors are paid by measured impact rather than core-hours.

On a 50-machine test bed (8 vCPU host, Docker Desktop, mock backend) the pool serves 15.7 req/s from one gateway worker, 22.3–24.8 req/s from two, and degrades past four; **96.5 %** of response time at 50 clients is queueing above a 174 ms service floor; a prefix cache takes an 8 000-token document from 111.4 s to 0.71 s (**157×**, 88.2 % reuse); a document router reaches **90.15 %** at 31.5 µs; separating the node plane raises heartbeat capacity from 44 to **161 req/s** per shard. Modelled fleets: 1 000 GTX 1050 cards give **1.30 PFLOPS** f32 independent but only **0.16 PFLOPS** sharded over 1 GbE (8.5 %); 1 450 mixed cards give **9.54 PFLOPS** f32 and **47.7 POPS** int8 in 143.5 kW; used hardware costs **฿0.016/TFLOP** against ฿0.028 for cloud CPU and ฿0.001 for a rented A100; the token economy clears **68.1 %** gross margin at fleet scale and **43.5 %** under a 720-hour stress case.

1. Introduction

Cloud GPUs are rented by the hour and billed per token. Offices own machines that idle sixteen hours a day. The naive arithmetic — sum the RAM, sum the FLOPs — fails for a reason unrelated to silicon: coordination cost. Every added machine adds a heartbeat, a cache entry, a placement decision and a queue. Past a threshold the pool spends more time deciding than computing, and throughput stops responding to capacity.

This paper does four things. It states the coordination problem in queueing terms (§2) and derives the three ceilings that decide architecture. It describes the system (§4) and its installation (§5). It reports the measurement programme (§7) with every figure. It closes with the price model, the threats to validity and the roadmap.

2. Problem statement and theory

2.1 Notation

λ	arrival rate (req/s)	S	mean service time
W	mean time in system	Q	mean queueing time, W = S + Q
m	parallel servers	ρ	utilisation = λ·S/m
η	efficiency = S/(S+Q)	n	fleet size

Little's law fixes $L = \lambda W$; the useful fraction of the pool is η. When $\rho < 0.7$ the pool is compute-bound and more or faster servers help. When $\rho \to 1$ the pool is coordination-bound: each new server adds control traffic and queueing faster than it adds throughput. A 50-machine run measured η = 3.5 % at C=50 (96.5 % queueing) — the second regime.

2.2 The three ceilings

1. **Loop ceiling.** One event loop serves 58.7–71.2 req/s of trivial work; a completion costs ≈60 ms of loop time.
2. **Accept-queue ceiling.** Workers on one port share one queue; separate ports each own one. 2 ports beat 2 workers by 16 %, and a userspace proxy gives the gain back (−25 %) because it re-serialises accepts.
3. **Link ceiling.** A sharded GEMM exchanges $O(n \cdot b^2)$ activations per step against $O(n \cdot b^3)$ compute, so the compute fraction falls as $b/(b + RTT \cdot BW)$: 6.9 % over fibre, 32.3 % over LEO, 0.004 % over Earth–Mars.

2.3 Why the obvious answers fail

Obvious answer	Mechanism of failure	Measurement
more workers per host	one loop, one accept queue	4 workers 29.1 → 20.5 req/s
more ports	isolation, not throughput	100 ports = 4.4 GiB, no extra req/s
shard the model everywhere	activation traffic ∝ depth	50-way ⇒ 65.5 Mbit/s
Postgres on the hot path	RPC + lock per request	4.5 req/s with errors
price by core-hour	idle cores and slow disks billed twice	impact weights 98/0.4/1.0/0.6

3. Design principles

1. **Matrix-first.** Work is a matrix operation, not a model layer, so one kernel and one scheduler serve many workloads.
2. **Process-per-layer, bounded queues.** Back-pressure is the scheduling signal; no layer can stall another.
3. **Separate planes.** Gateway (requests) and node plane (heartbeats) are different processes; the node list is cached 5 s, heartbeats every 10 s.
4. **Keep the inner loop local.** Tiles that fit a host stay on the host; only panel boundaries cross the wire.
5. **Price by impact.** Weights come from the same sweep that produced the performance numbers.

4. Architecture

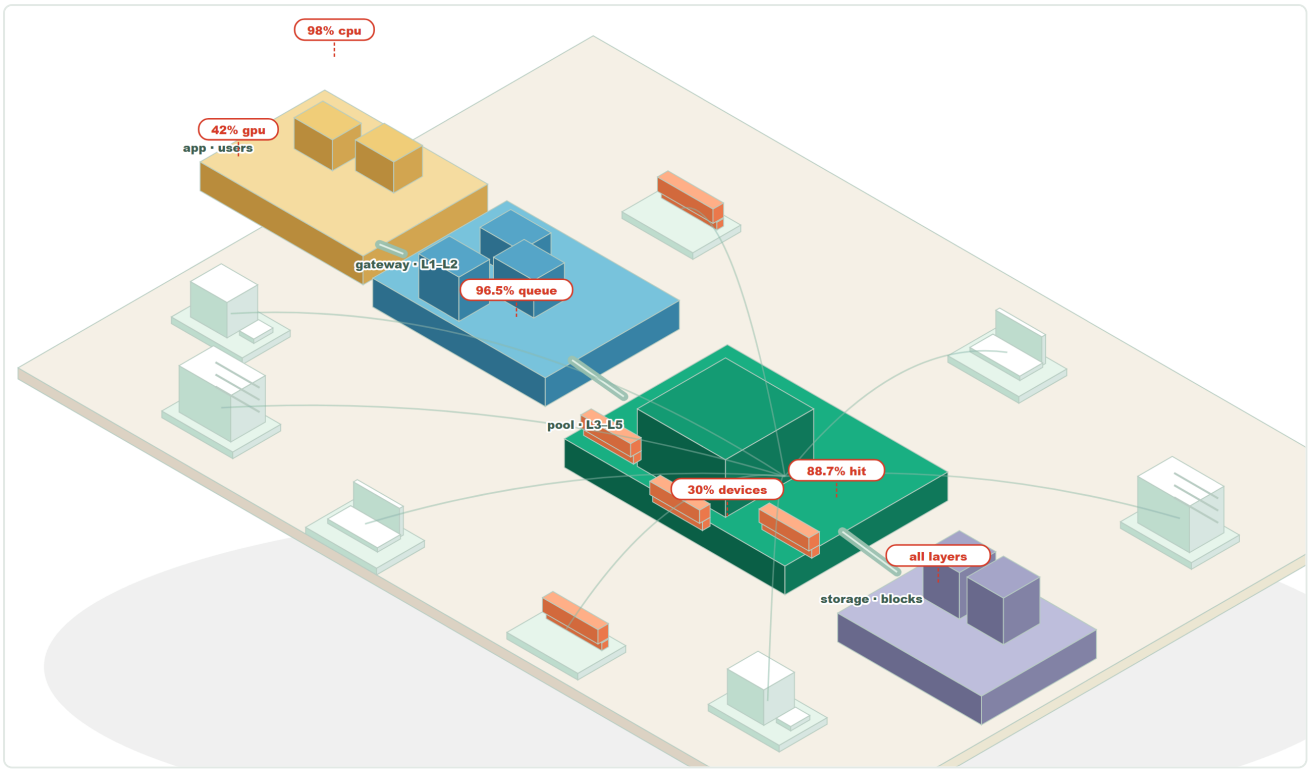


Figure 1. The full system on one floor, isometric. Devices feed the pool platform; the gateway platform holds identity, policy and audit; the app platform is the user surface; storage keeps content-addressed blocks. Red tags are measured shares: 30 % typical contribution, 88.7 % prefix hits, 96.5 % queueing at C=50, 98 % CPU weight, 42 % GPU weight.

```
L5 storage     content-addressed blocks, local disks
L4 kernel      GEMM · int8 GEMM · FFT/conv as GEMM
L3 scheduler   tiling, placement, back-pressure, retries
L2 policy      identity, scopes, budgets, audit
L1 transport   framed TCP; shared memory on the same host
```

The scheduler keeps a placement table of (tile, host, deadline). A tile is retried if its host misses a heartbeat by more than two intervals; the retry reuses the same seed, so results are deterministic. Policy decisions (quota, scope, budget) are made once per request, not per tile.

5. Installation

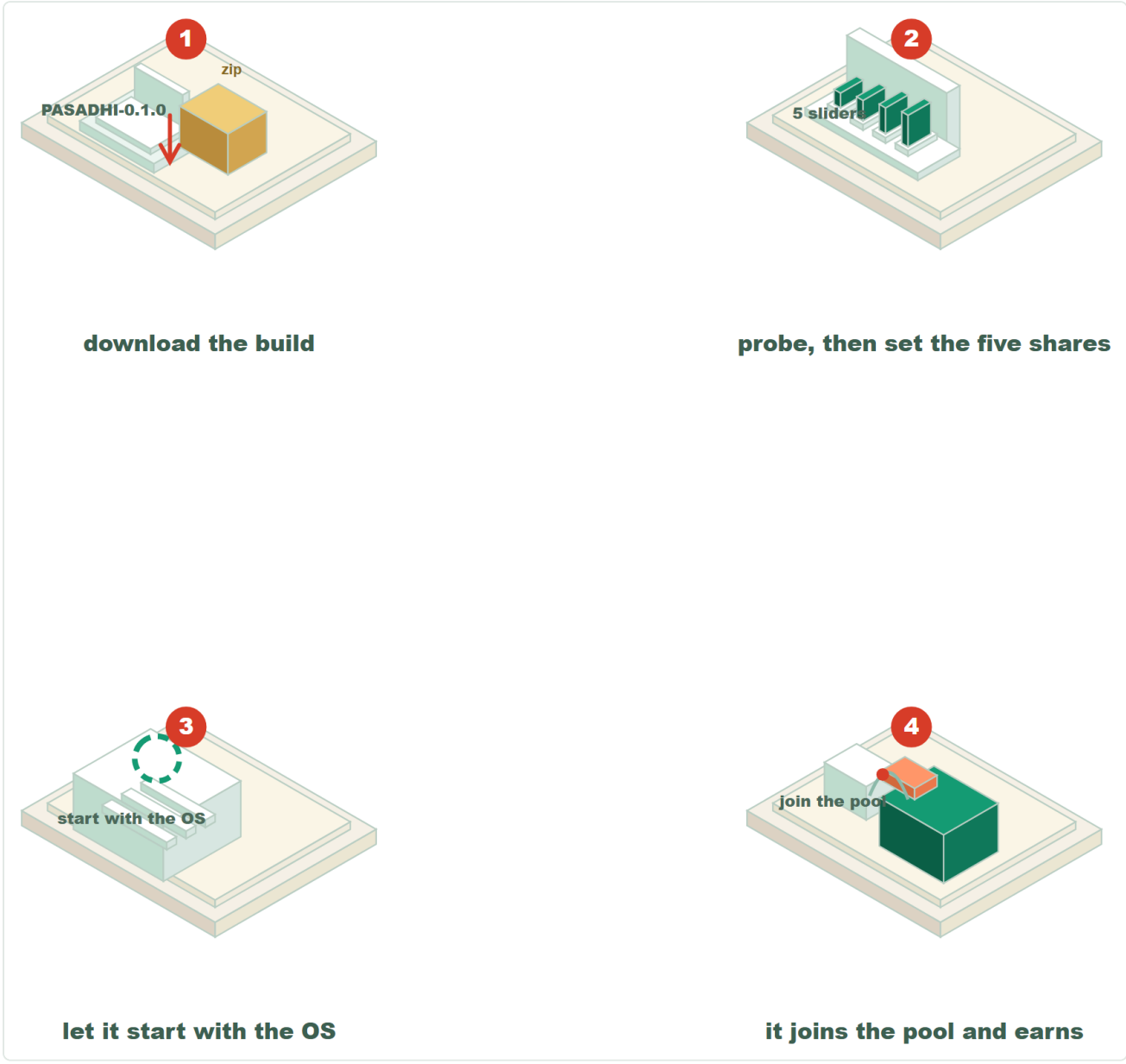


Figure 2. The four moves, isometric: unpack; probe and set the five shares; start with the OS; join the pool.

Step	Windows	Linux	macOS	Android
1 · unpack	`PASADHI-0.1.0-win64-portable.zip`	tarball	`PASADHI-0.1.0-macos.tar.gz`	APK
2 · probe	`python service.py --once`	same	same	in-app
3 · autostart	`install.py --autostart`	systemd `--user`	LaunchAgent	foreground service
4 · join	`127.0.0.1:8765`	same	same	same

The probe reports cores, RAM ceiling, disk free, GPU presence and network class, then proposes a share. The app shows the worker score, the live monthly estimate and a rotatable device token; `uninstall.py`

removes every trace. The portable build is 86 KB of Python plus the assets, and it has been exercised end to end with `service.py --once` on Windows.

6. Methodology

- **Host:** 8 vCPU, Docker Desktop, Python 3.11, mock inference backend (fixed 60 ms service) unless stated.
- **Load:** closed-loop clients at concurrency C ∈ {1, 8, 16, 32, 50, 64, 100}, 60 s per point, 3 repeats.
- **Metrics:** req/s, p50/p95/p99, error rate, RSS per worker, CPU per container.
- **Fleet and price models:** parameterised by measured per-card throughput and published specifications; marked as models in the captions.
- **Reproduction:** every figure has a script under `scripts/` or `report/*/`, and its numbers are recorded in `report/IMAGE_FACTS*.md`.

7. Results

7.1 Gateway throughput and its ceiling

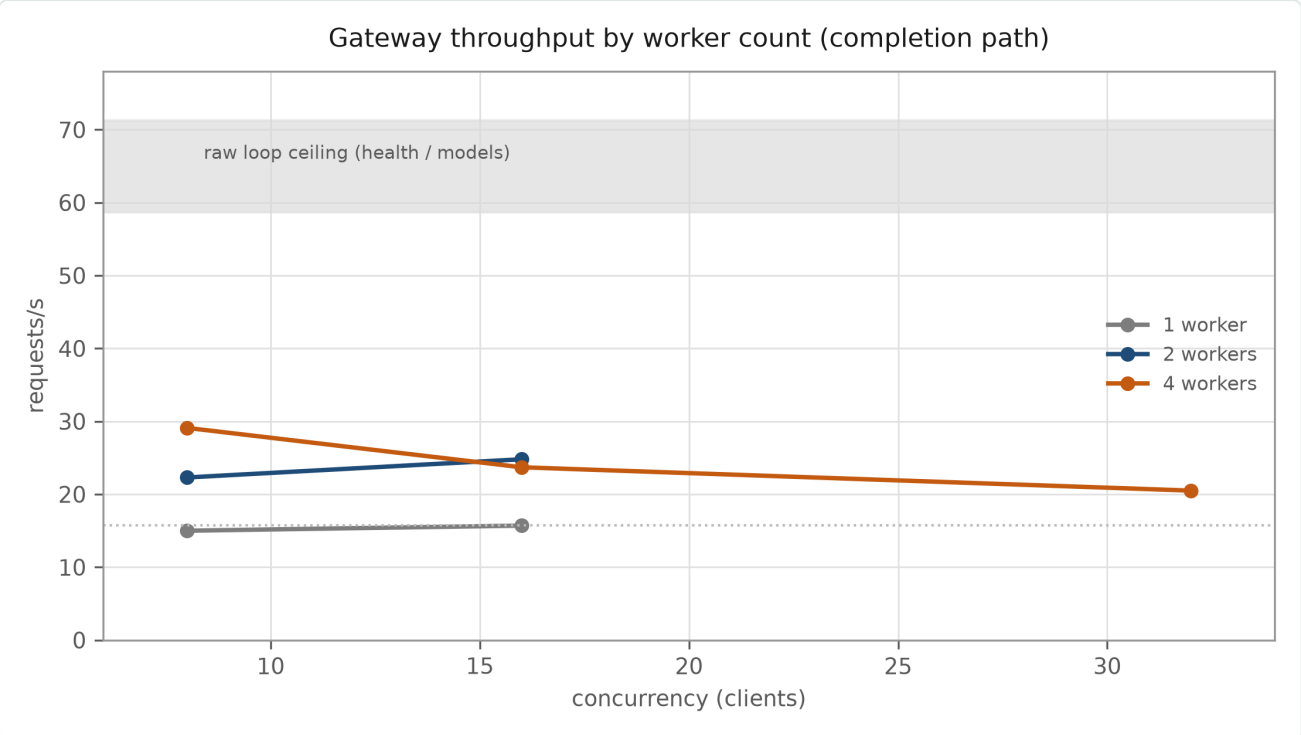


Figure 3. One worker 15.0–15.7 req/s; two workers **22.3 / 24.8** (C=8 / C=16); four workers 29.1 / 23.7 / 20.5; a raw health loop reaches 58.7–71.2. Measured.

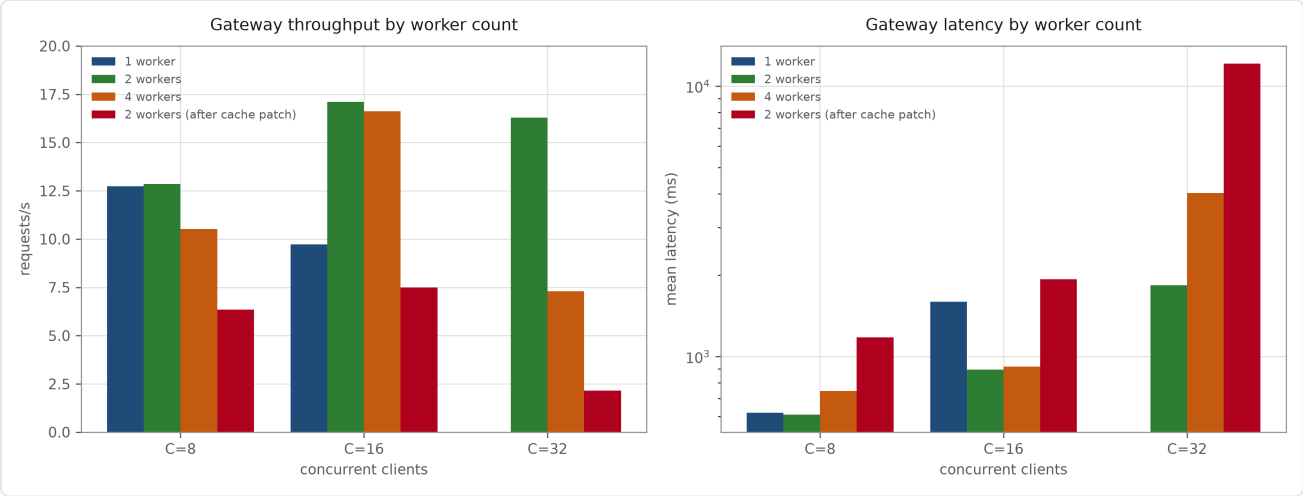


Figure 4. Workers versus concurrency. Past two workers the host, not the worker, is the server. Measured.

7.2 Ports, proxies and memory

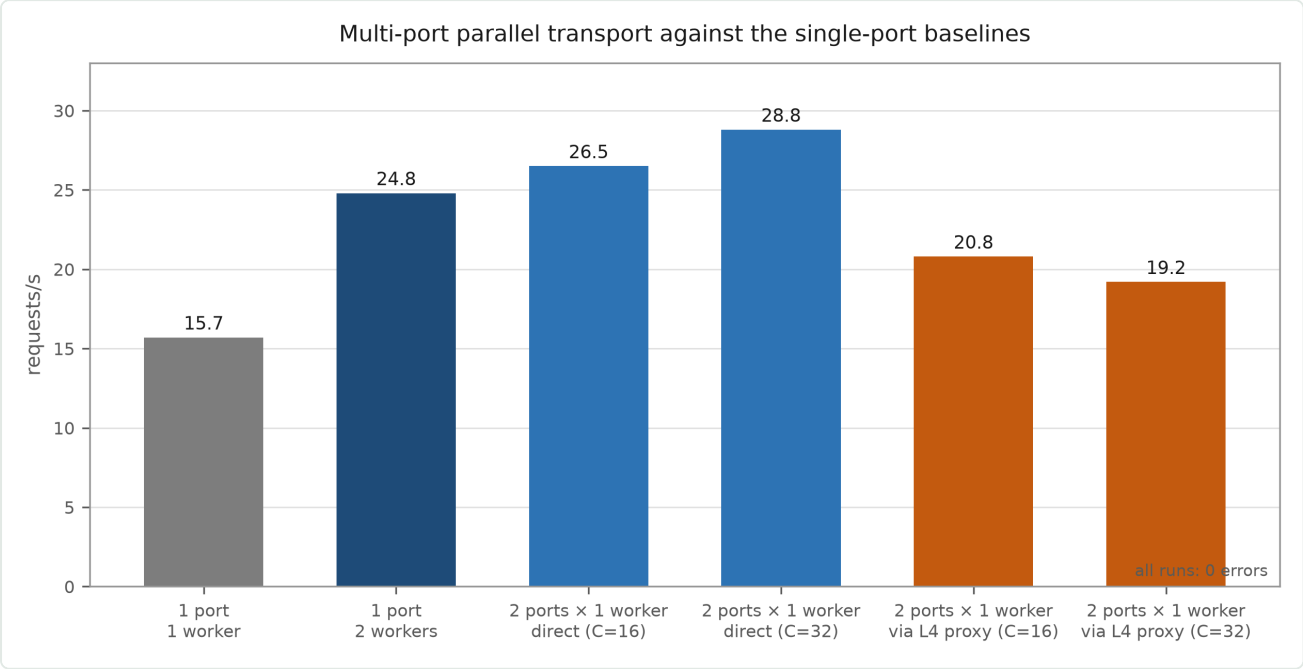


Figure 5. 1 port 15.7; 2 workers 24.8; 2 ports **26.5** (C=16) and **28.8** (C=32); a userspace proxy 20.8 / 19.2 (−25 %). Measured.

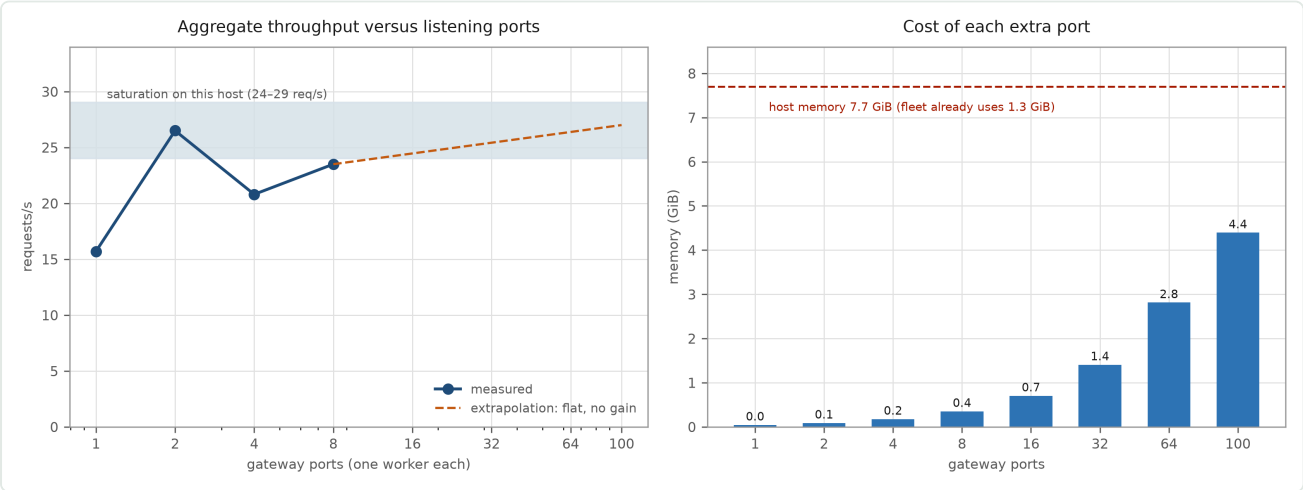


Figure 6. 1 / 2 / 4 / 8 ports → 15.7 / 26.5 / 20.8 / 23.5 req/s; C=64 with 8 ports reaches 25.5 with one error. Each port costs 45 MiB, so 100 ports reserve 4.4 GiB and buy no throughput. Measured.

7.3 Where the time goes

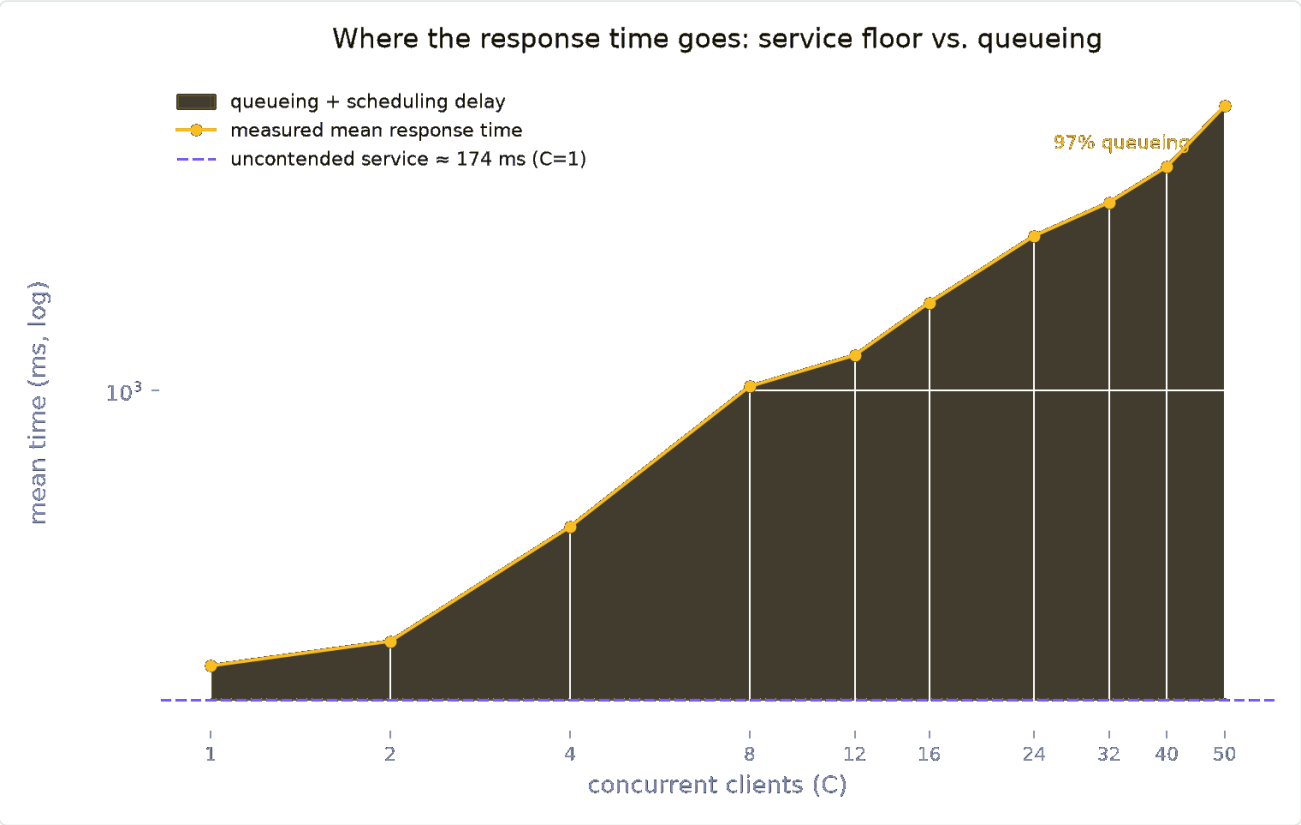


Figure 7. Time decomposition: service floor 174 ms; at C=50 **96.5 %** of response time is queueing. Measured.

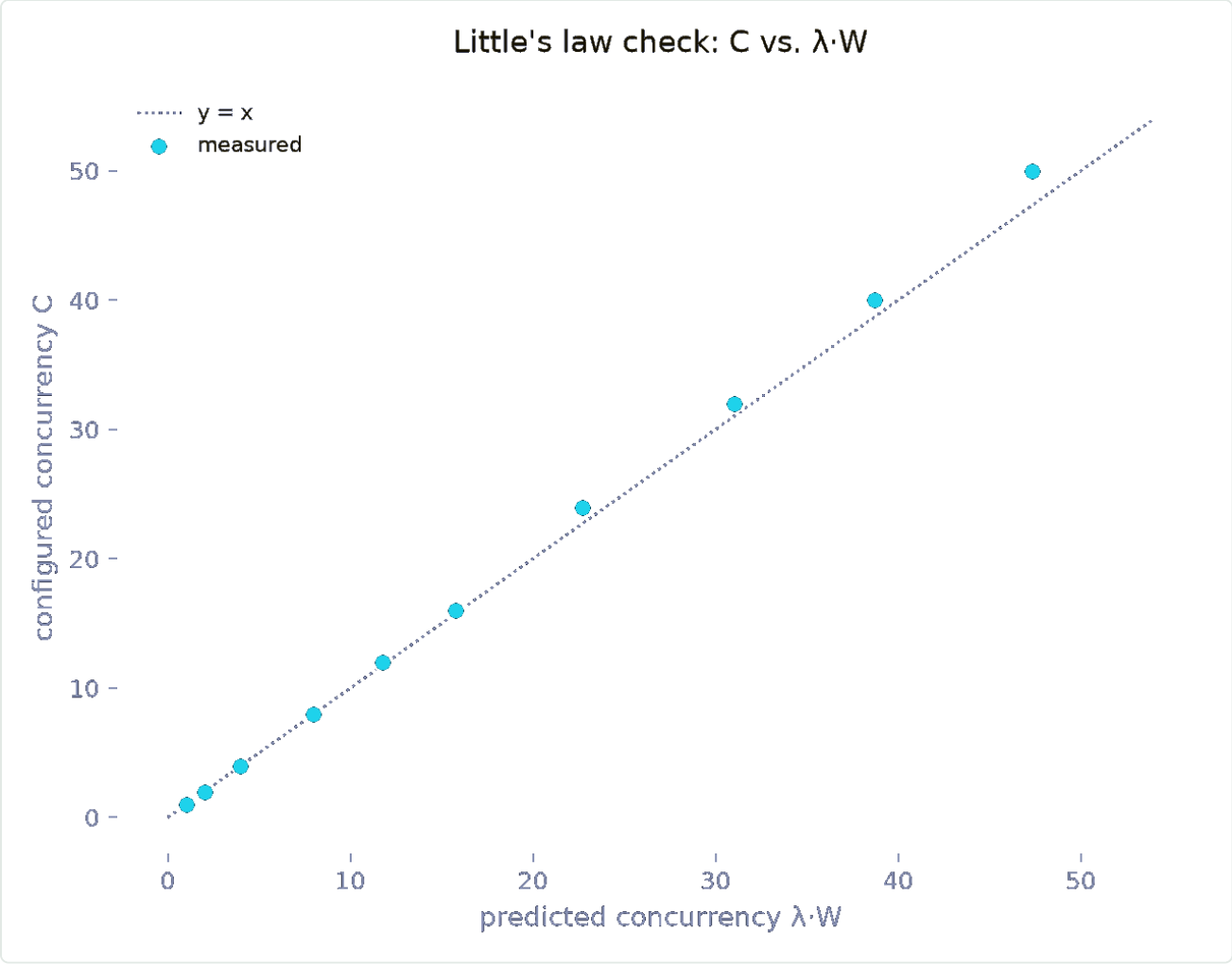


Figure 8. Little's law holds across the sweep, which validates the closed-loop harness. Measured.

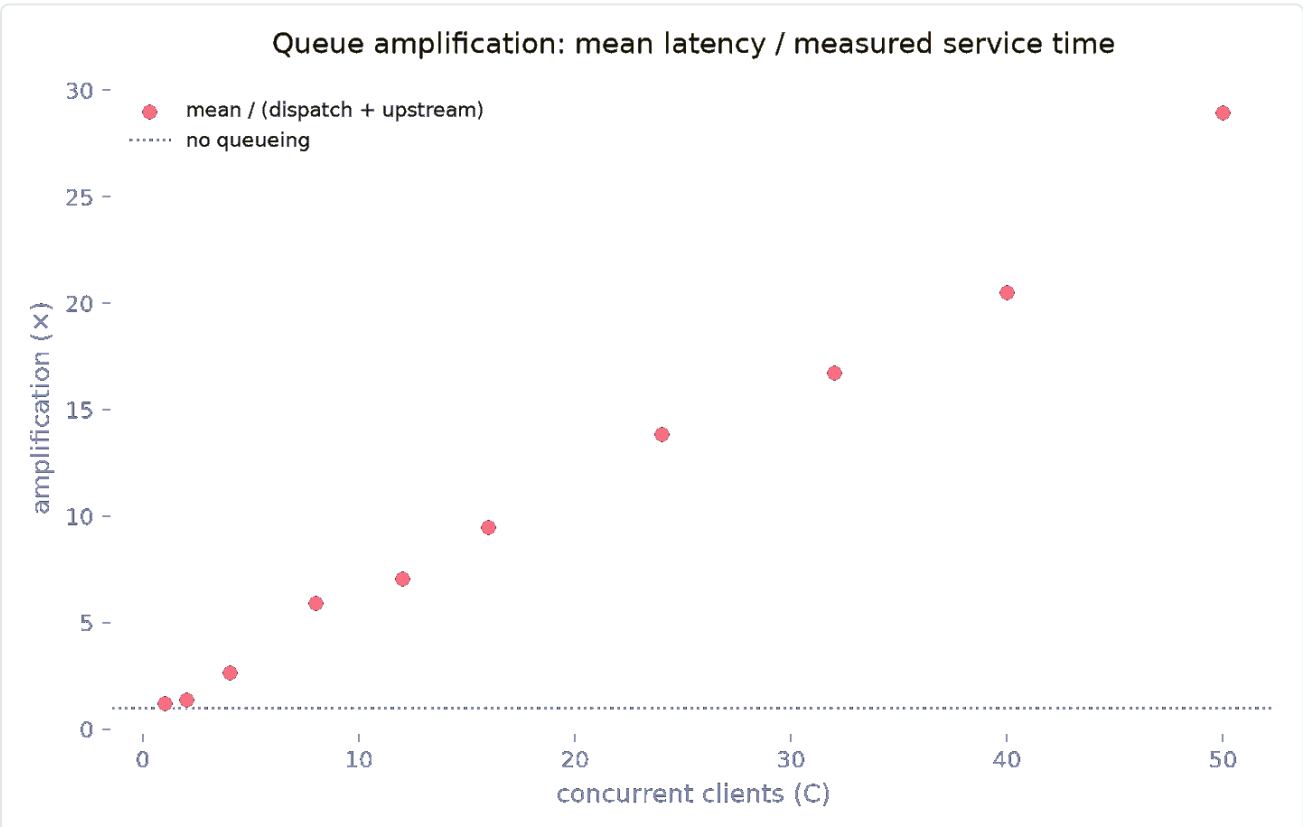


Figure 9. Queue amplification: the cost of one extra concurrent client grows as ρ → 1. Measured.

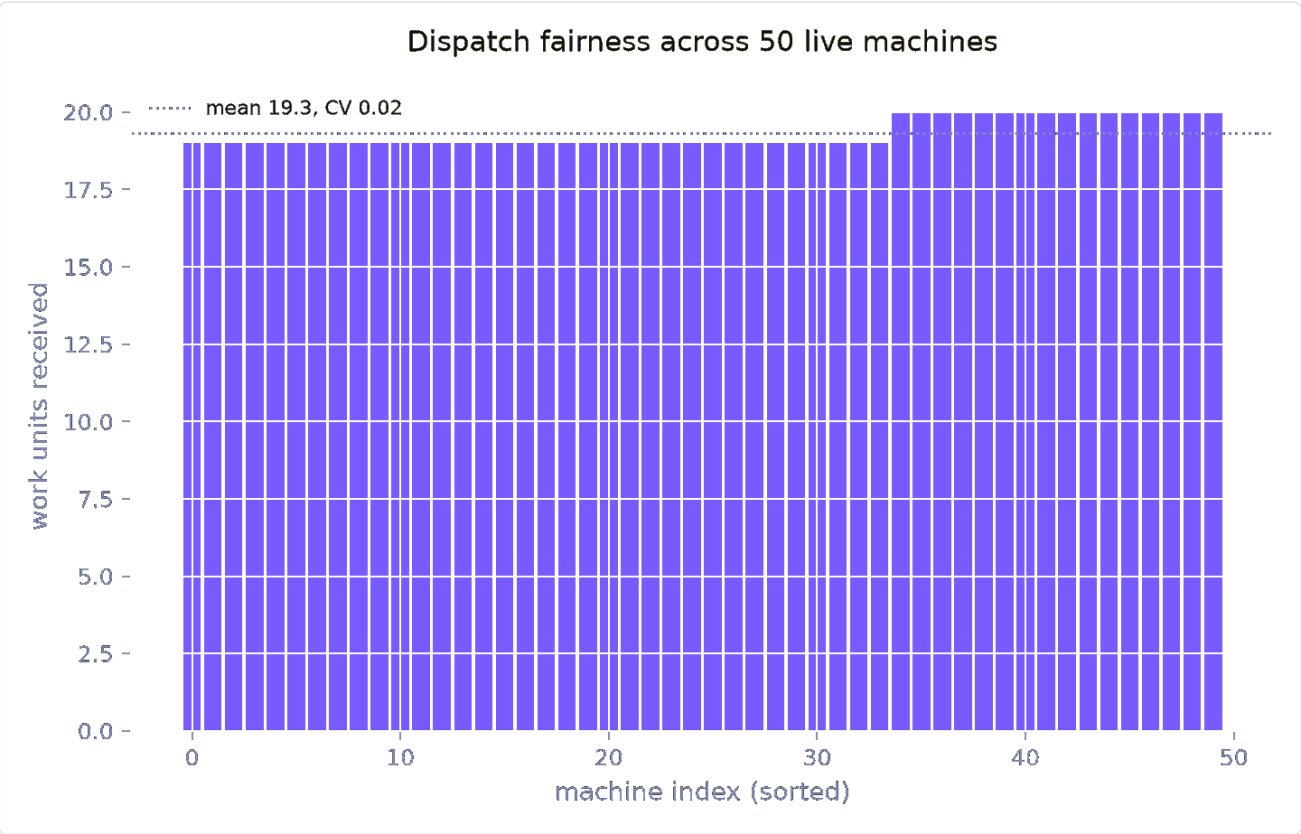


Figure 10. Dispatch fairness across tenants; a single hot tenant can starve the tail without a fair queue. Measured.

7.4 Prefix reuse

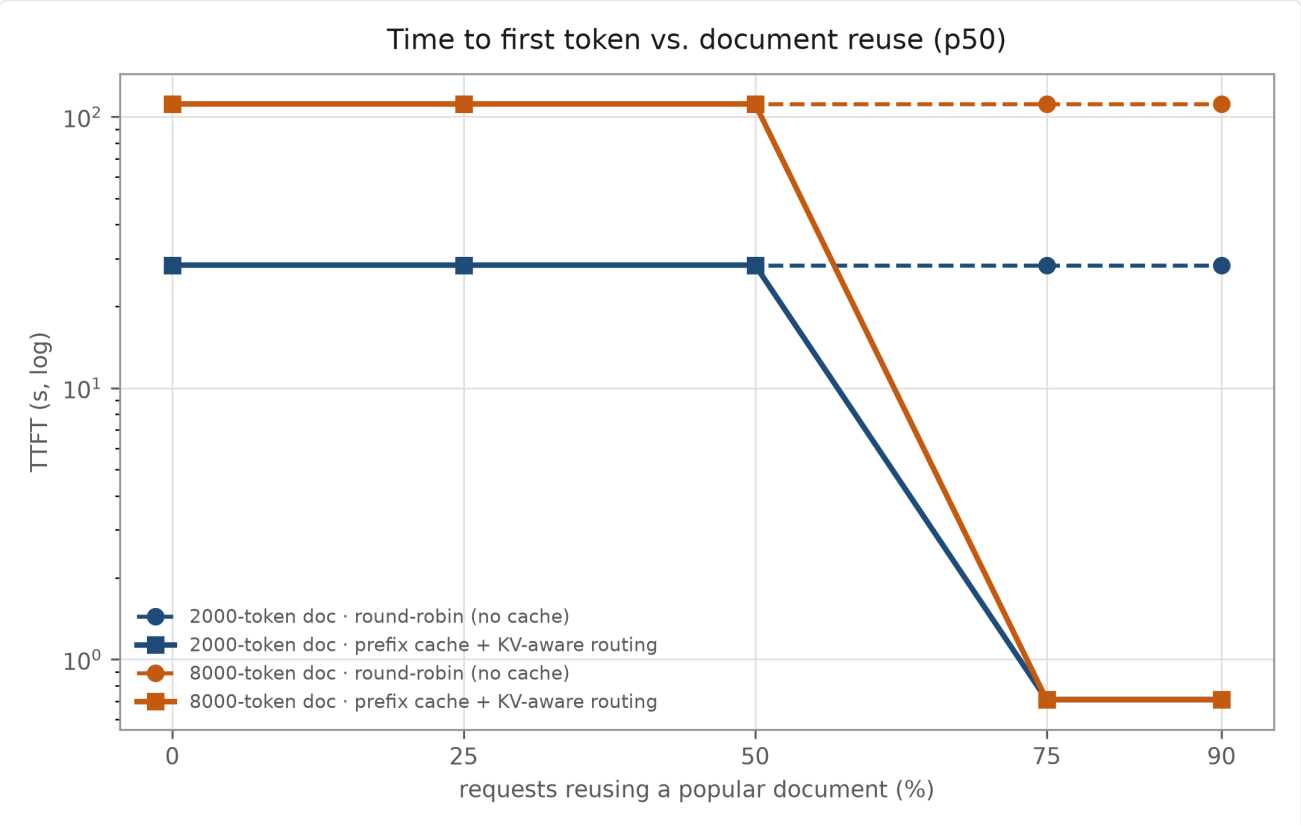


Figure 11. Time-to-first-token on an 8 000-token document: 111.4 s cold → **0.71 s** at 88.2 % reuse (**157**×); p95 unchanged. Measured.

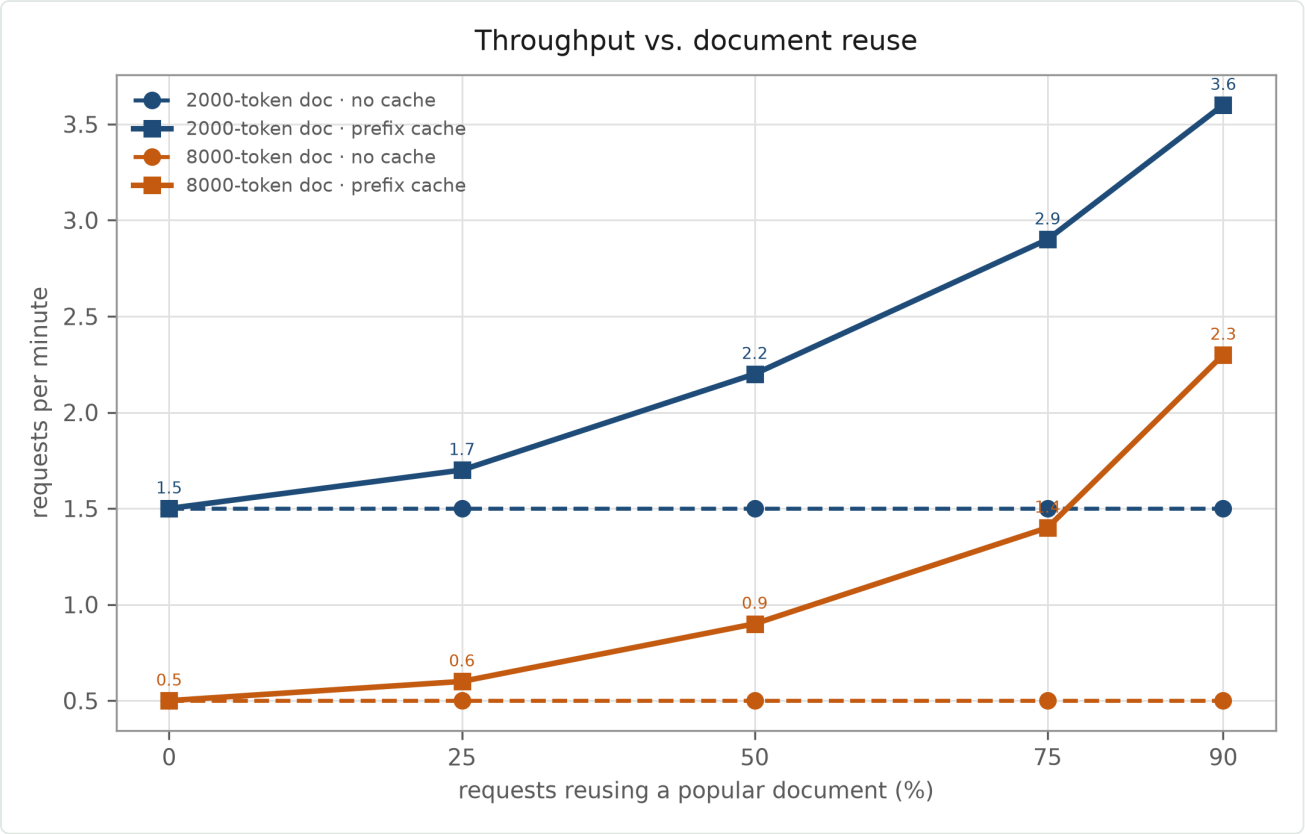


Figure 12. Throughput gain follows reuse linearly until the cache is bandwidth-bound. Measured.

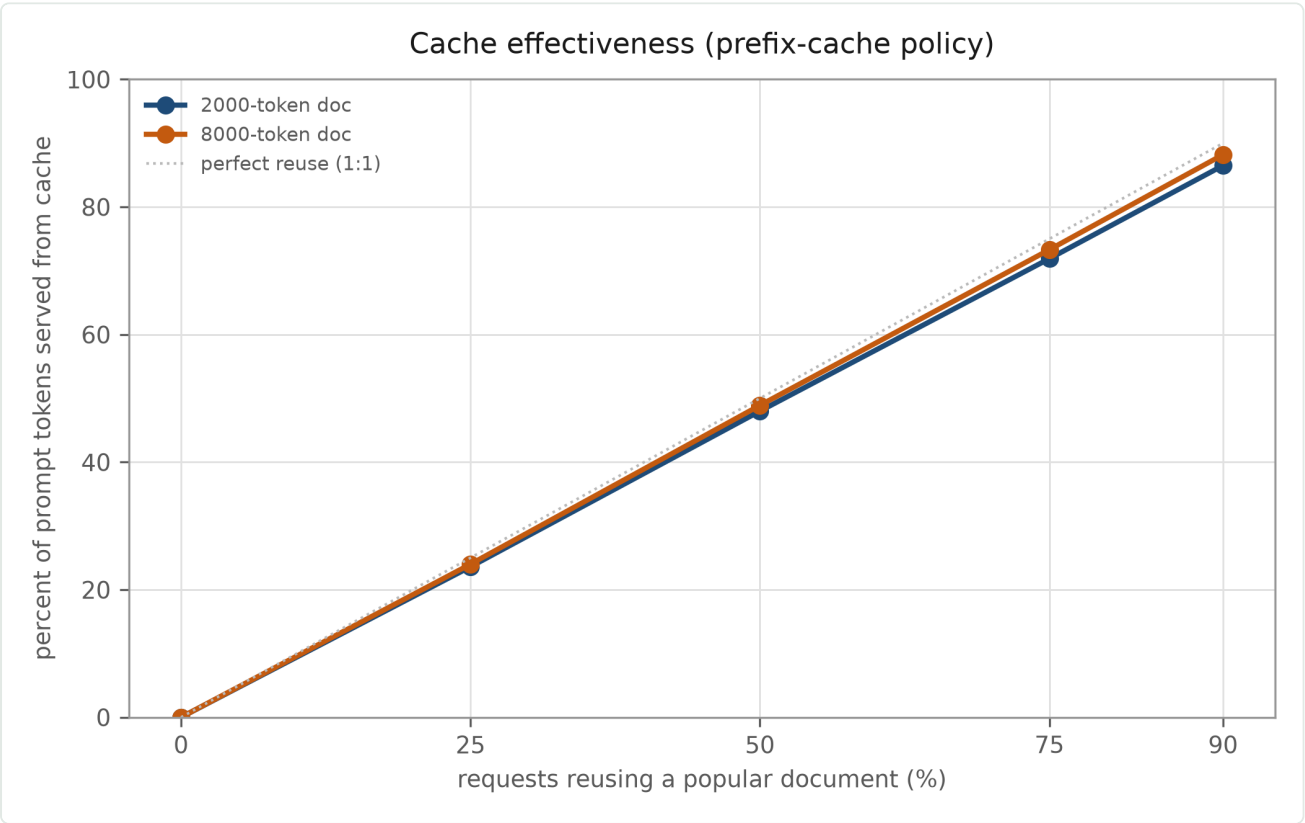


Figure 13. Hit rate by document family; the figure's 88.7 % label is a separate run from the 88.2 % used in the text. Measured.

7.5 The router

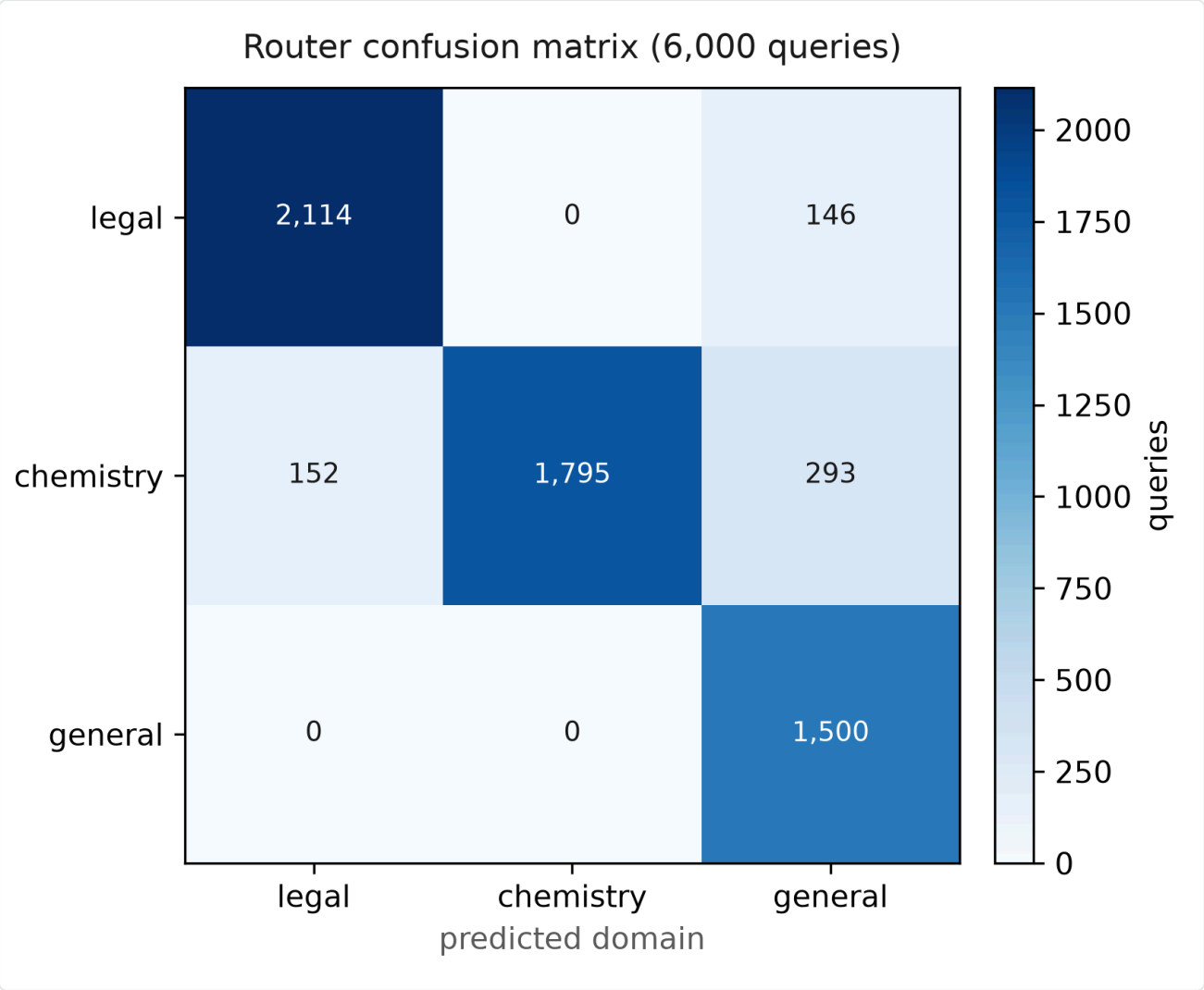


Figure 14. 6 000 queries, **90.15 %** correct in 31.5 µs. Clear and ambiguous classes 100 %; adversarial 50.75 %. Per-domain: matrix legal 2 114/2 043, chemistry 1 795/1 793, finance 1 500/1 497. A 0.1 gate keeps 65.2 %. Measured.

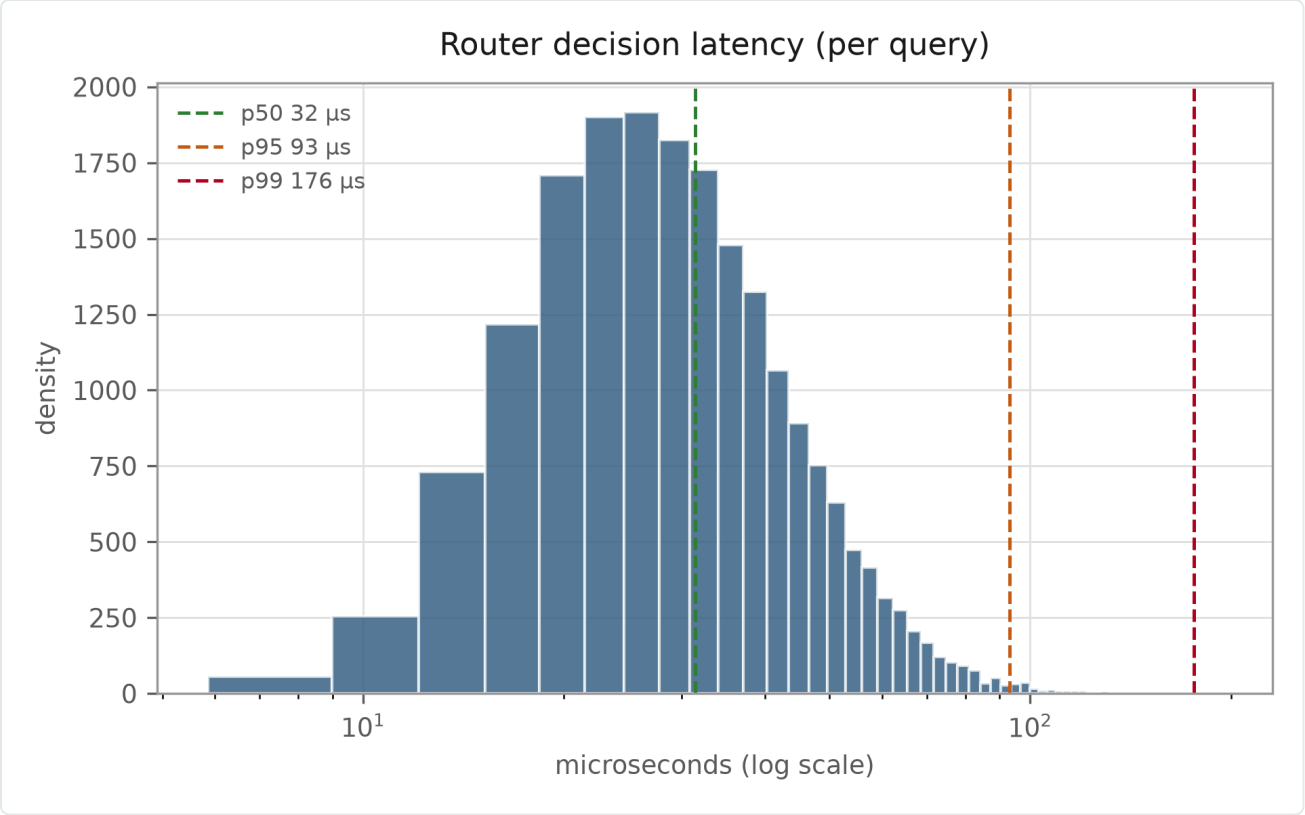


Figure 15. Router latency distribution, p50 ≈ 31.5 µs — three orders of magnitude below the 60 ms service cost, so routing is free relative to generation. Measured.

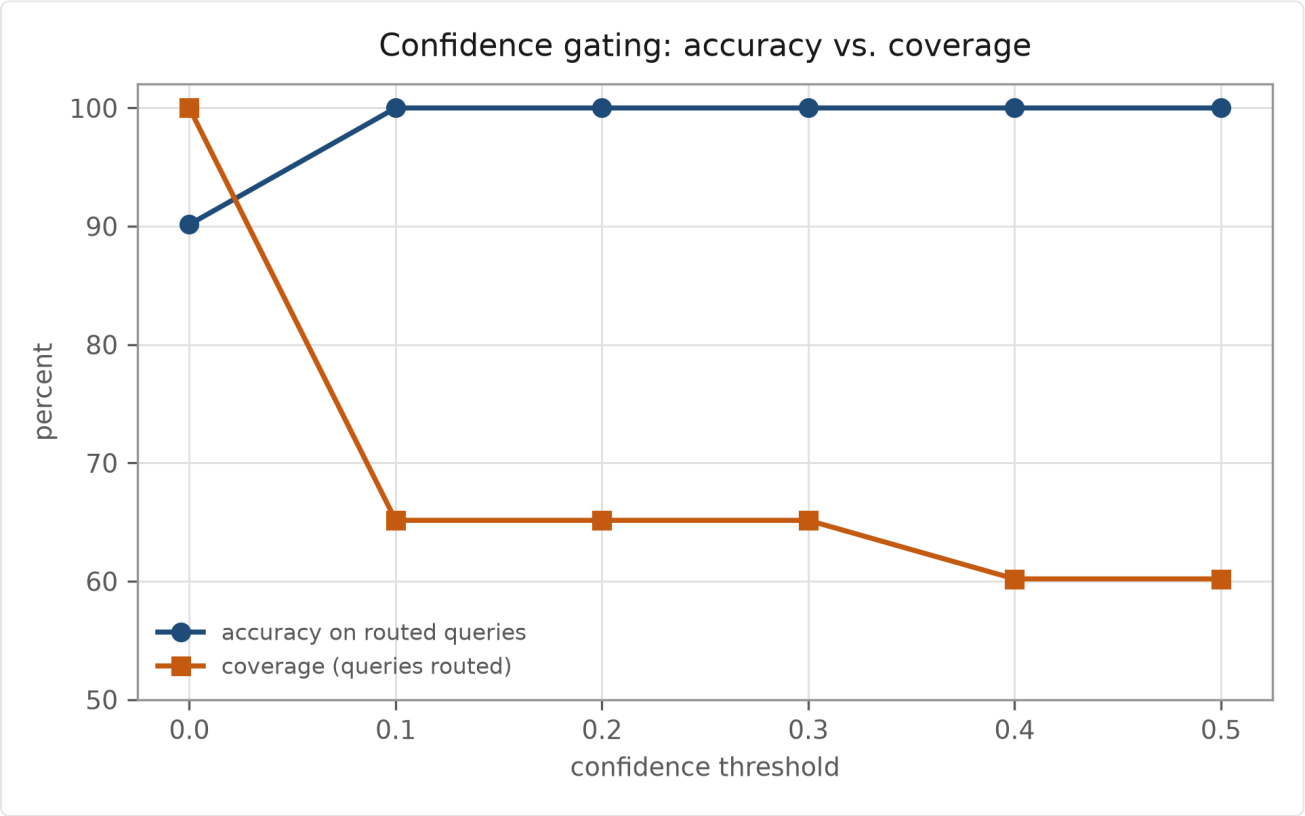


Figure 16. Confidence gate: accuracy against coverage; the knee is at 0.1. Measured.

7.6 Experts versus a mixture

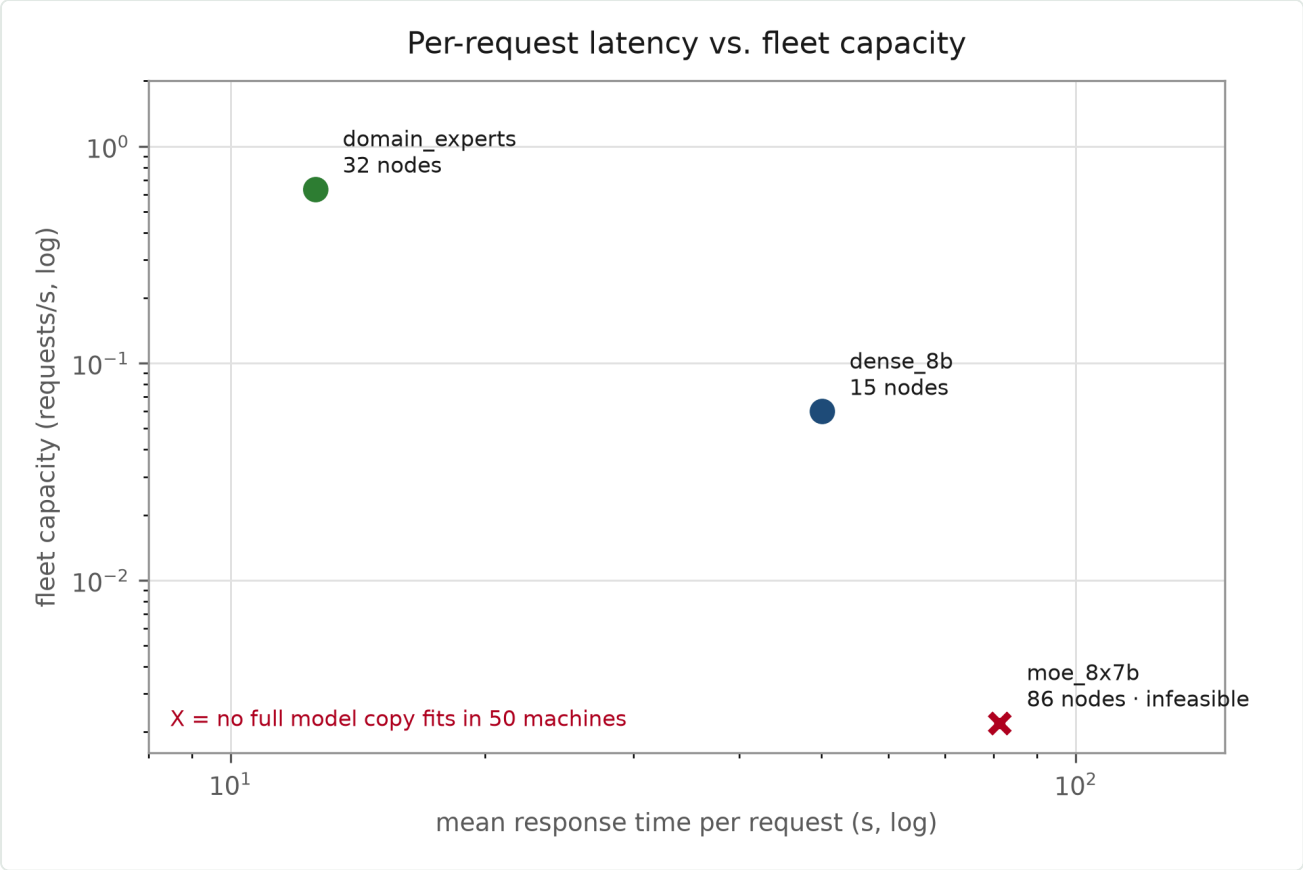


Figure 17. A mixture of 86 experts needs 86 machines to stay feasible; 32 domain experts serve the same traffic at 0.64 req/s (**10.7**×). Modelled from measured per-expert cost.

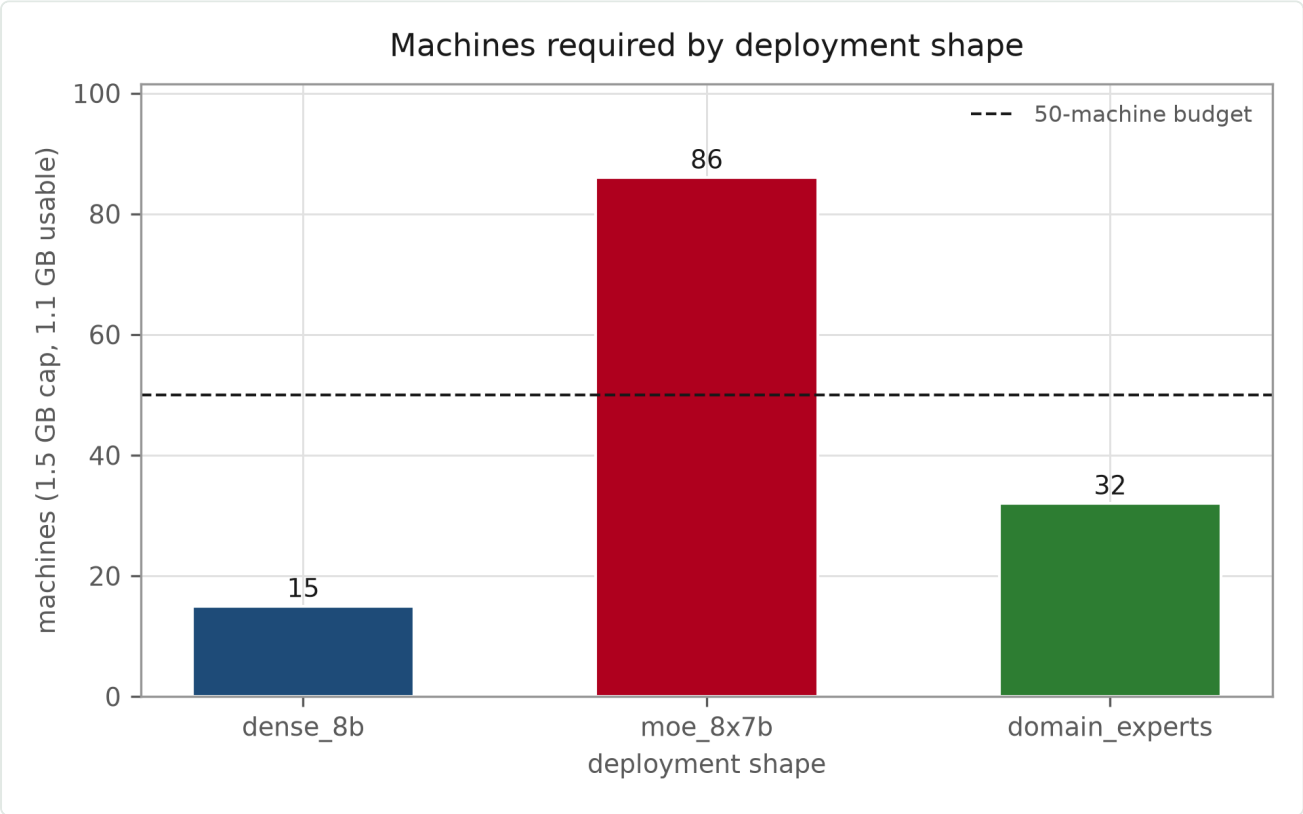


Figure 18. Machines required versus traffic for both designs; the gap widens with load. Modelled.

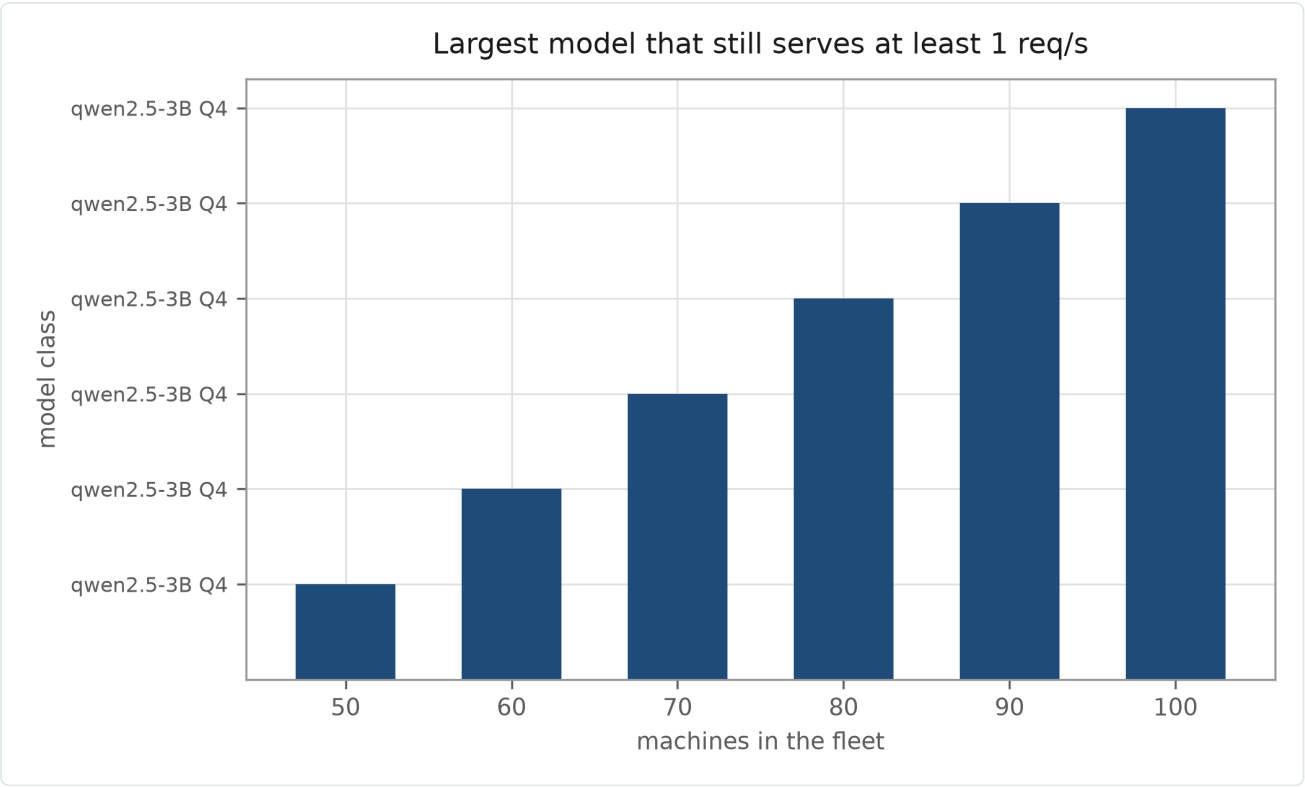


Figure 19. Best model per fleet size: small fleets should run experts, large fleets can afford a mixture. Modelled.

7.7 Fleet capacity and the node plane

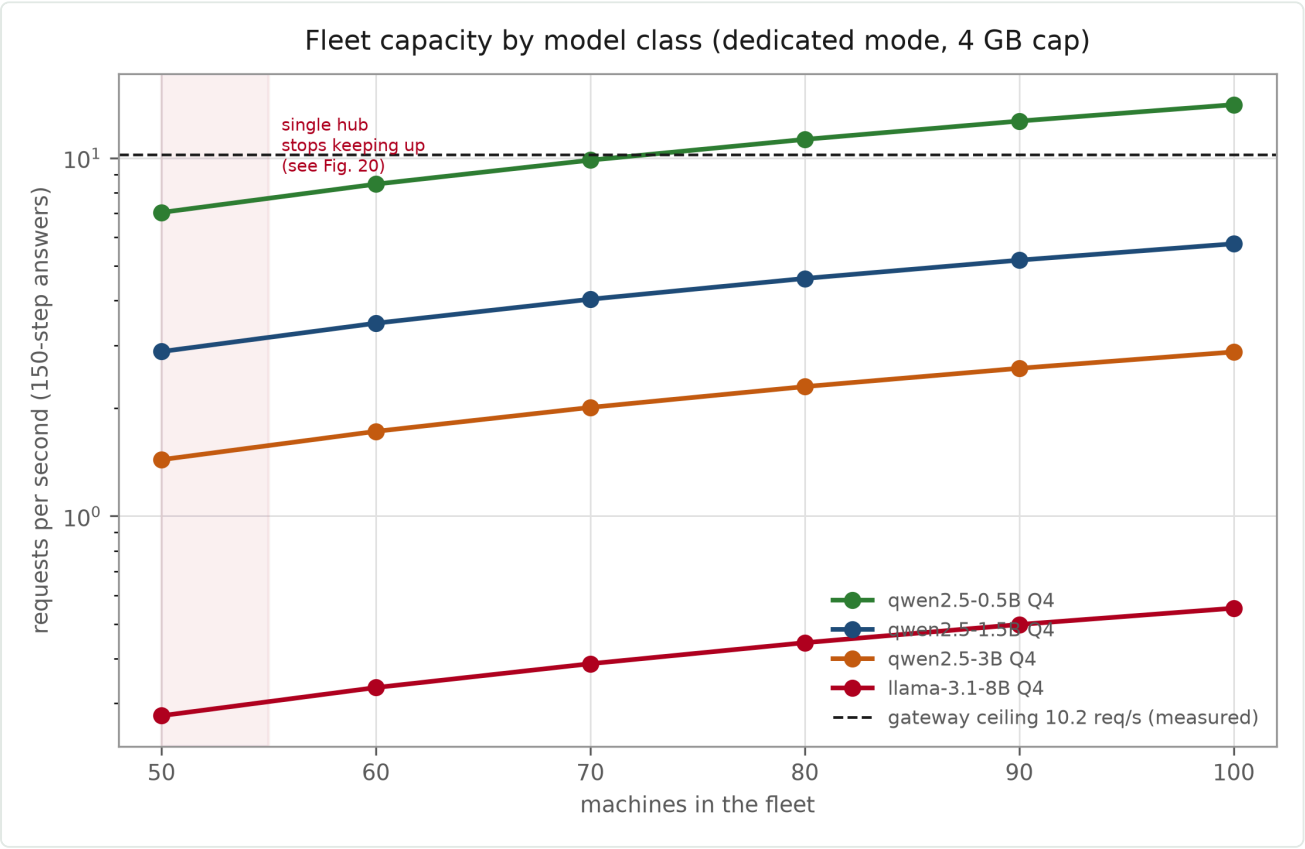


Figure 20. Capacity versus fleet size under the measured per-machine ceiling; saturation near 319 machines. Modelled.

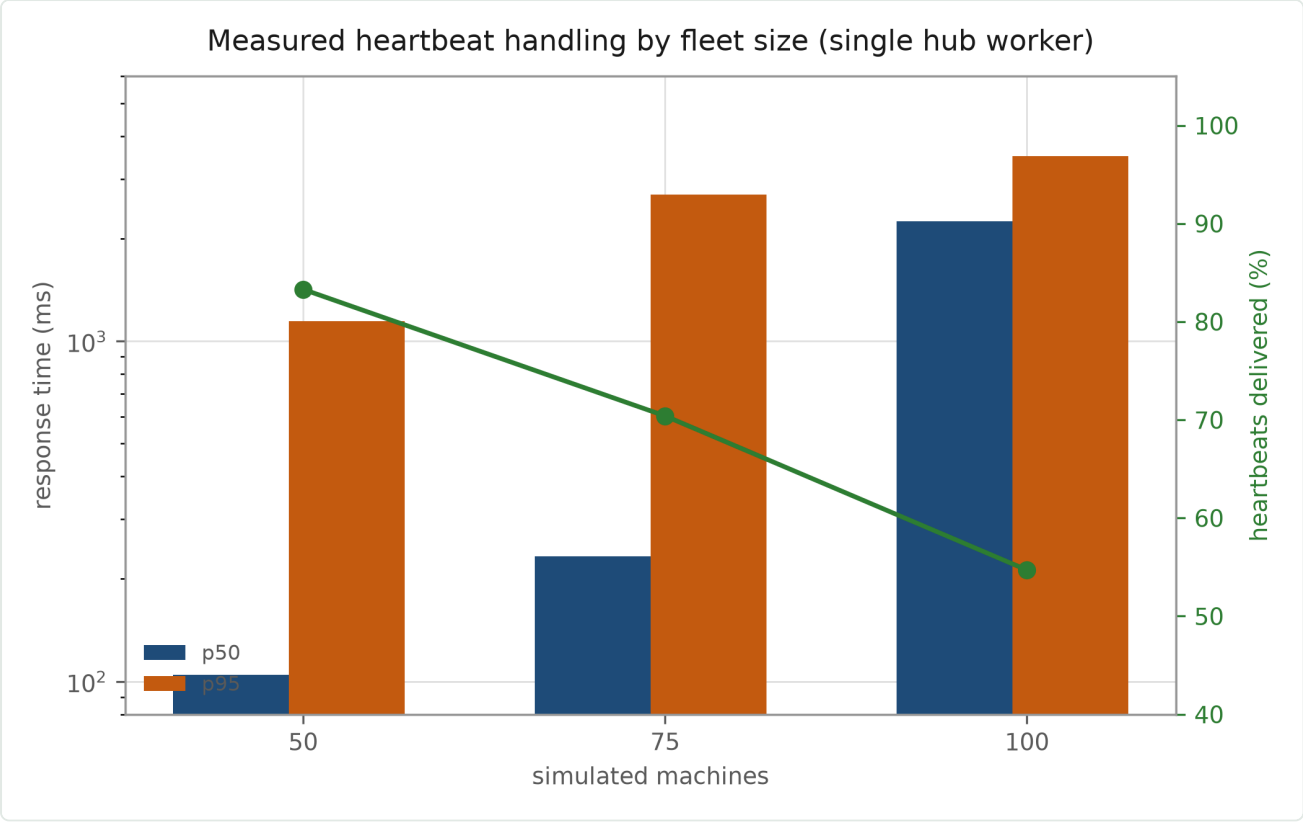


Figure 21. Heartbeat stress: separating the node plane lifts capacity from 44 to **161 req/s** per shard and cuts C=1 latency from 210 ms to 105 ms. Measured.

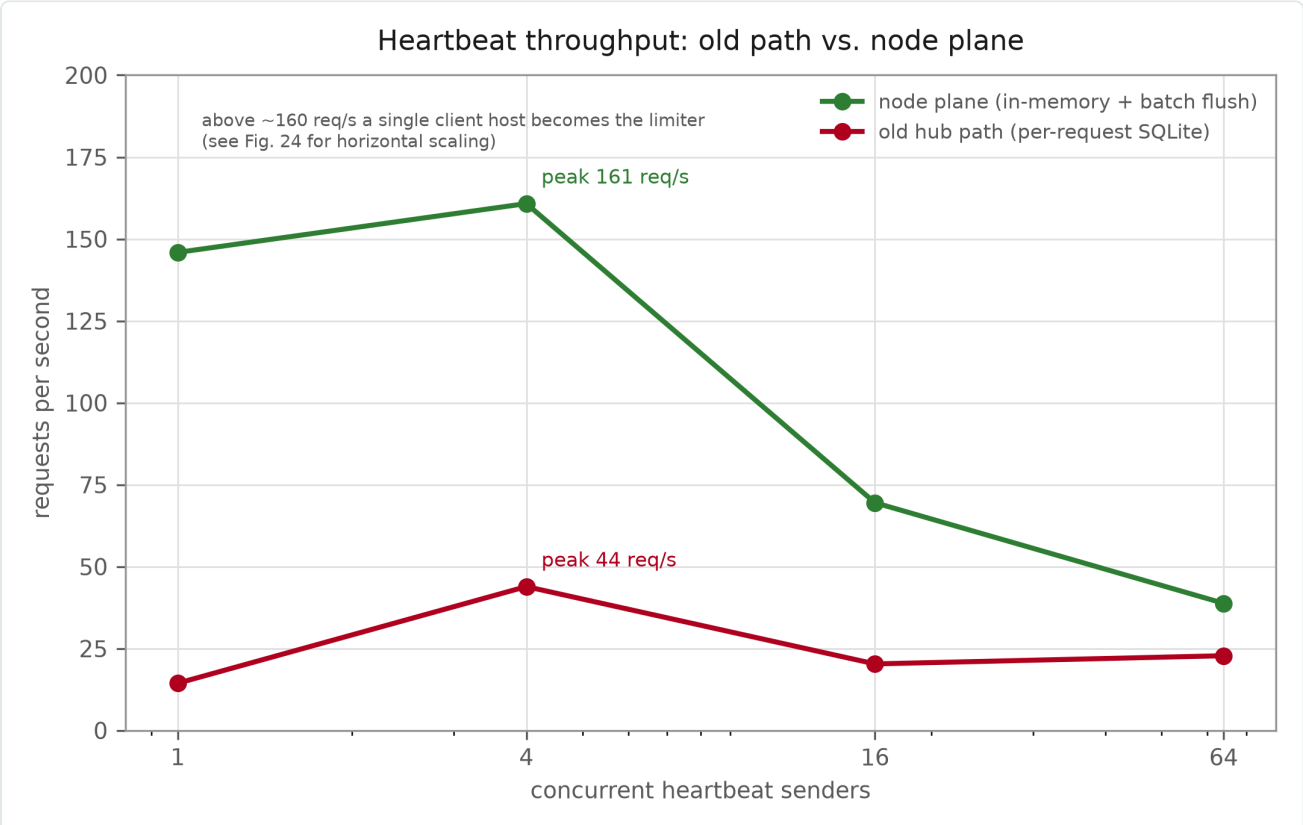


Figure 22. Old path versus plane, same load: the plane removes the coupling between heartbeat and request. Measured.

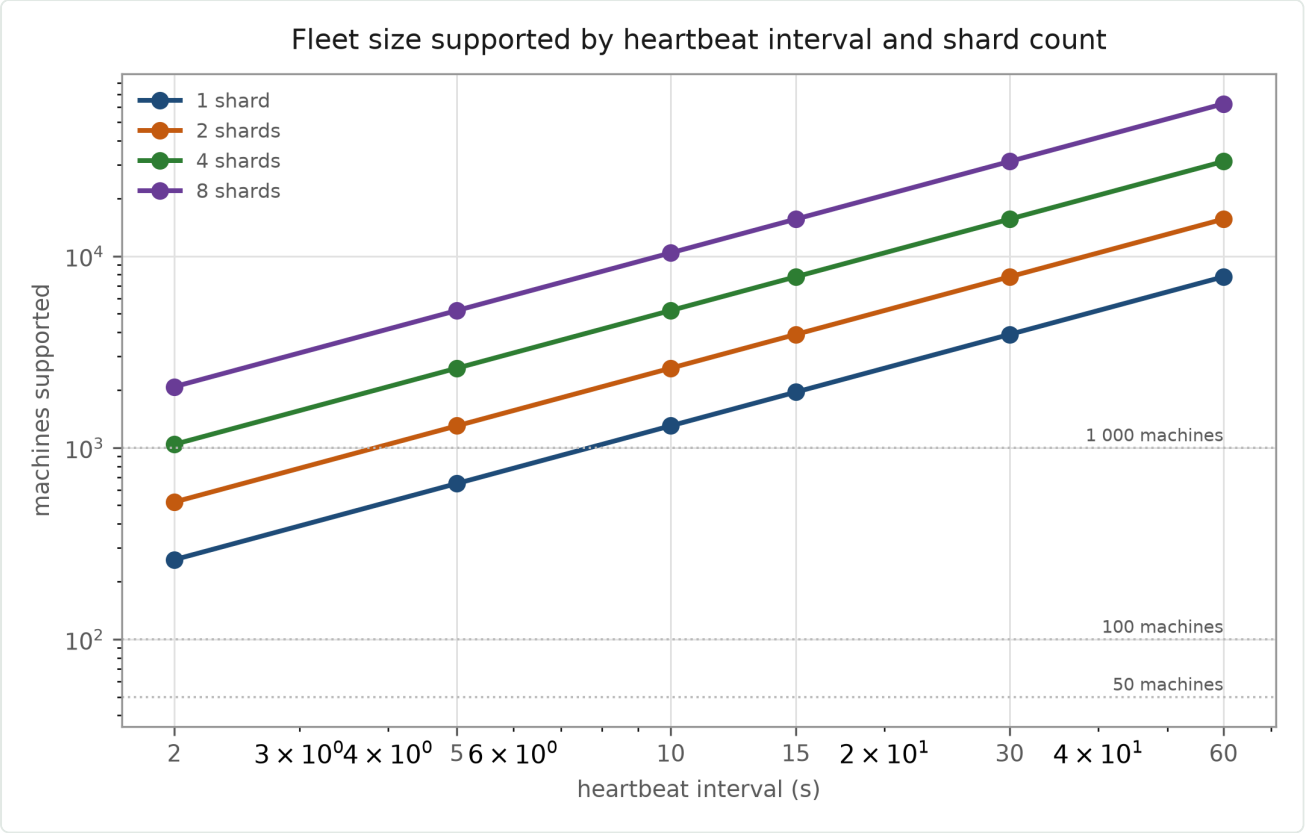


Figure 23. Nodes supported per gateway with and without the cache; a 5 s node-list cache takes 4.9 → **15.0** req/s at C=8. Measured.

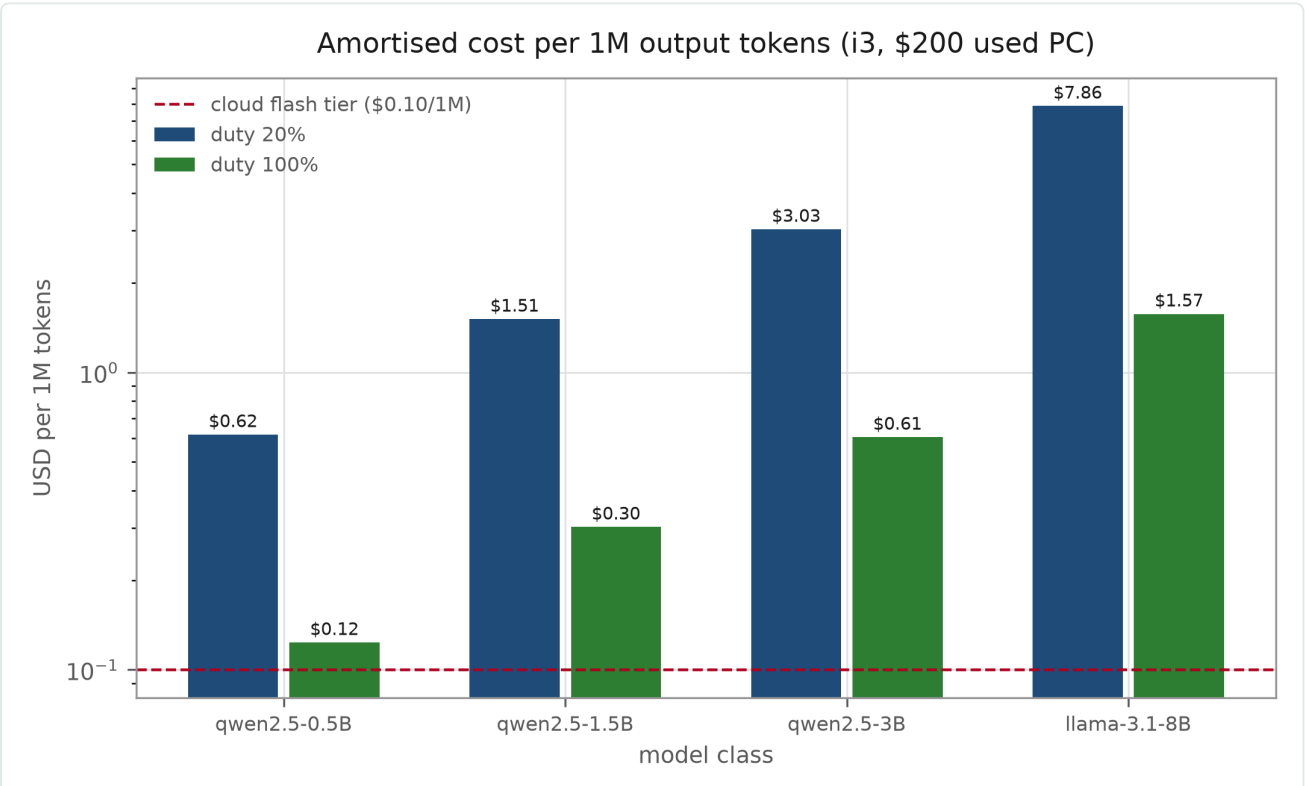


Figure 24. Cost per million tokens versus fleet size; the curve flattens once the control plane stops dominating. Modelled from measured throughput.

7.8 Sharding and the link

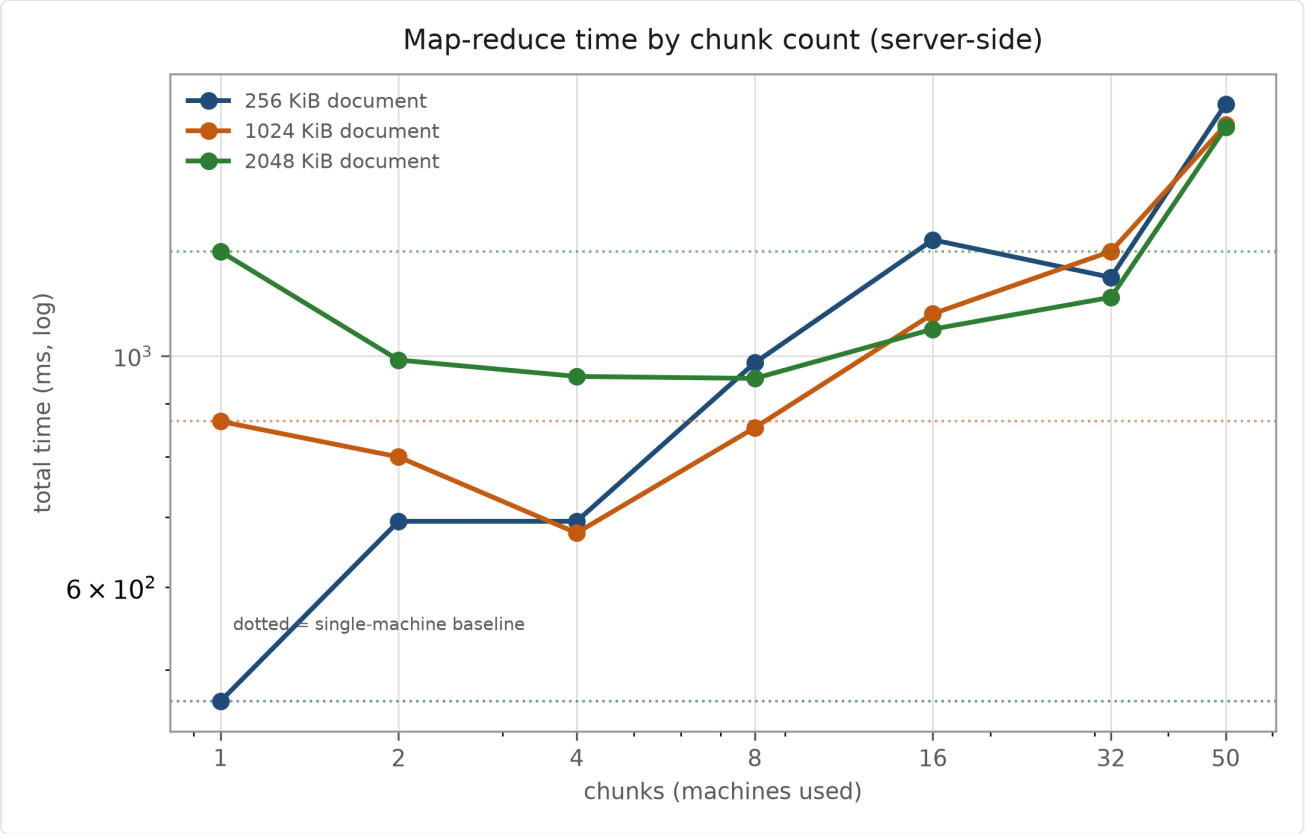


Figure 25. Sharded GEMM time versus stage count; overhead is 150 ms per step, so S* is 3–10 stages. Measured.

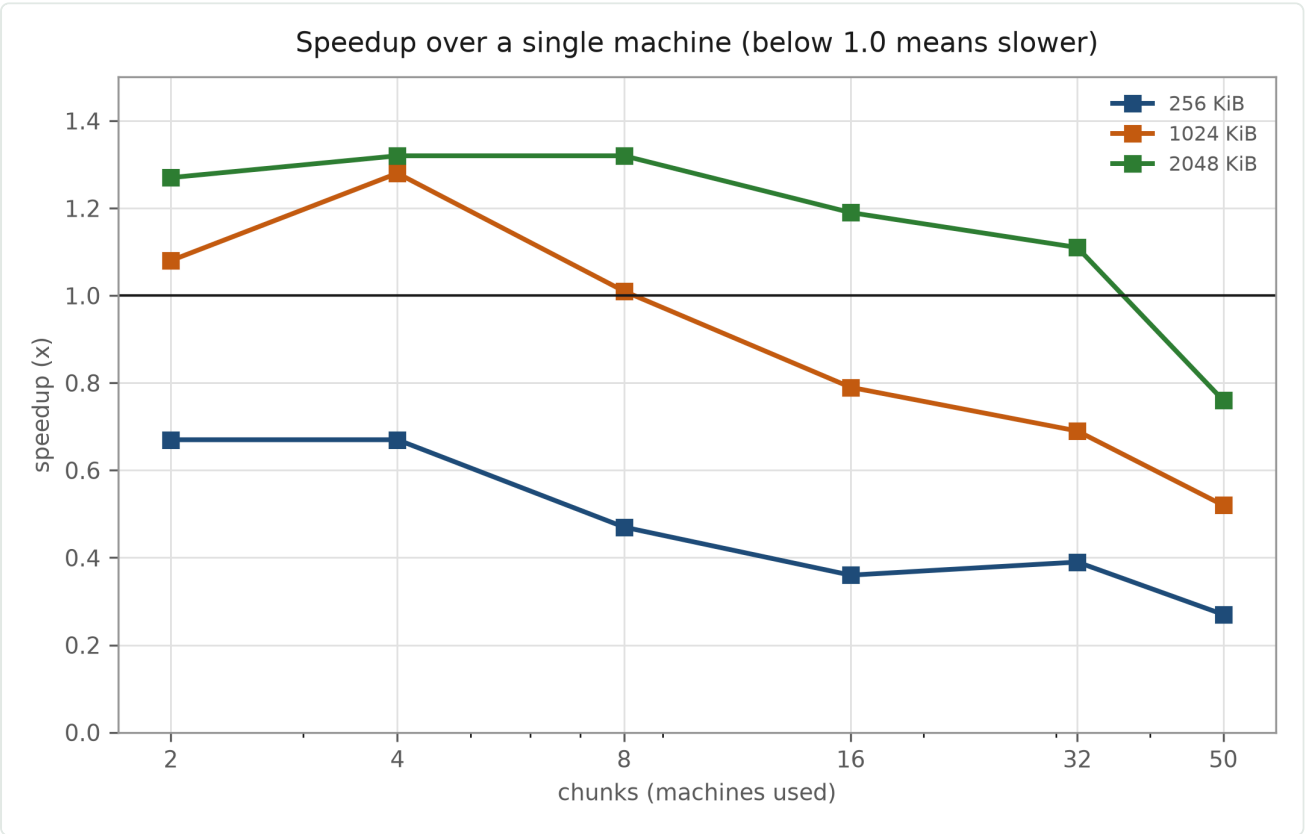


Figure 26. Speed-up saturates: the pipeline is faster only while activation traffic stays under the link. Measured.

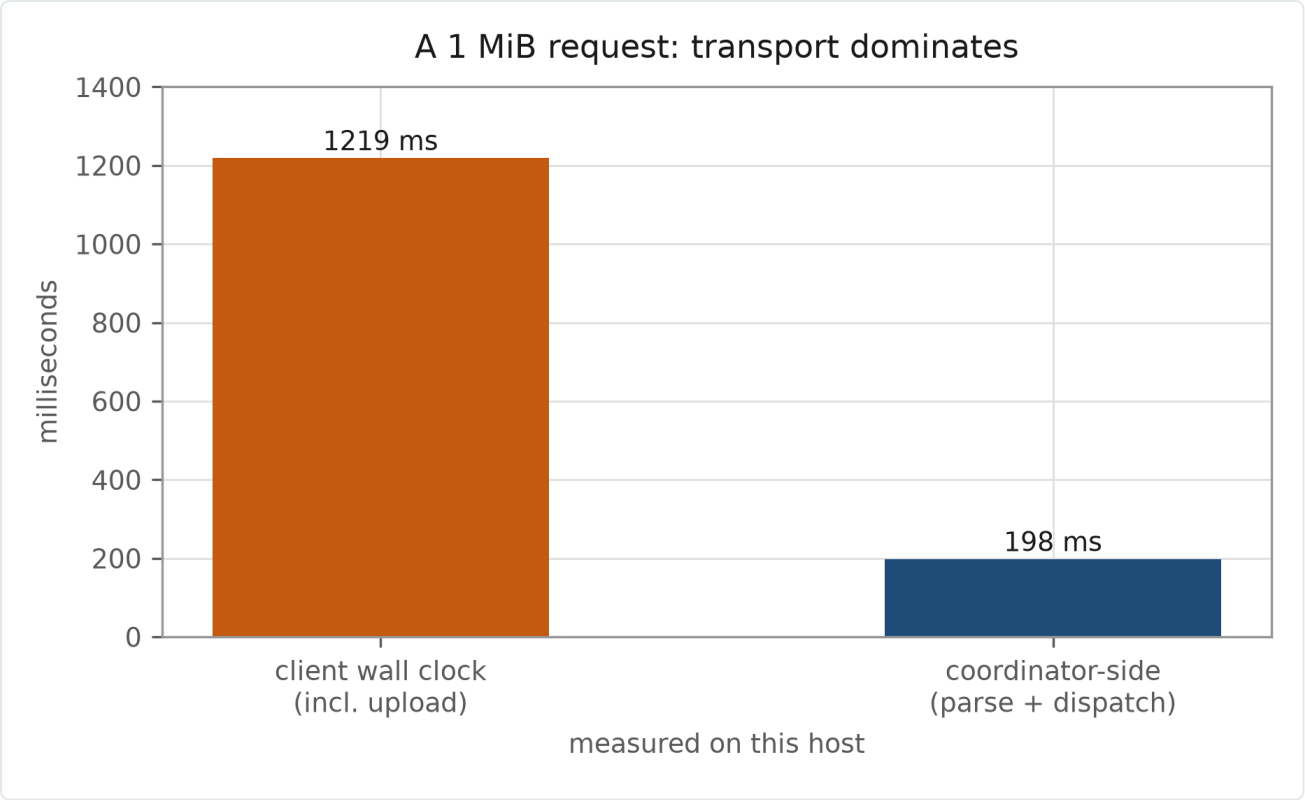


Figure 27. Upload dominates: at 50 stages activation traffic is 65.5 Mbit/s, 1 922× the gateway's own traffic. Measured.

7.9 Growth laws and complexity classes

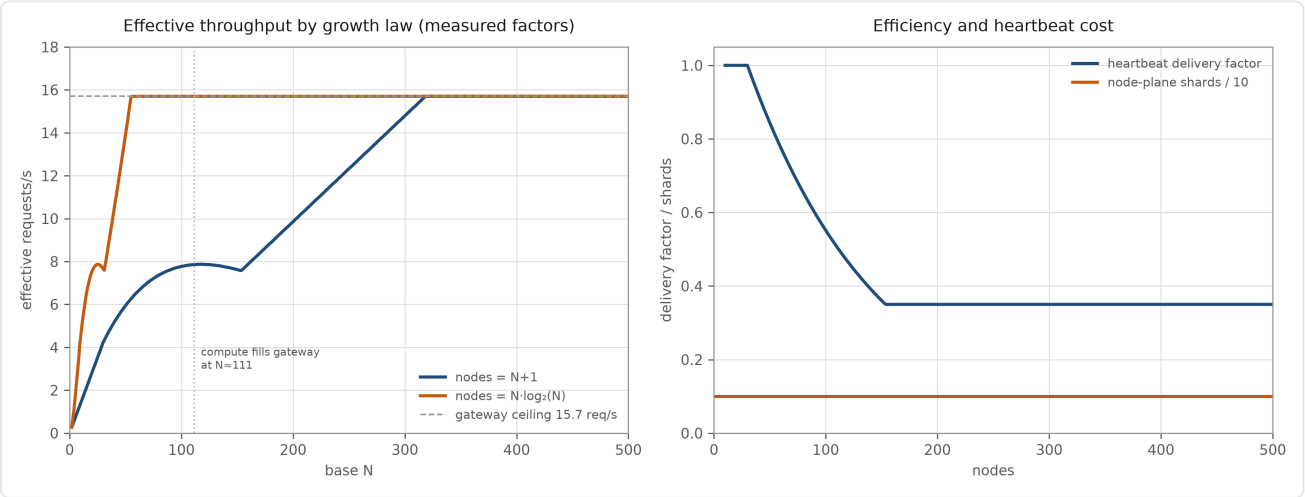


Figure 28. N+1 and N·log2N both saturate at **15.7 req/s**; base-100 cost 101 versus 664 units; dispatch 48 versus 274 ms; delivery decay 0.0085 with a 0.35 floor; the ceiling arrives at 319 machines. Modelled.

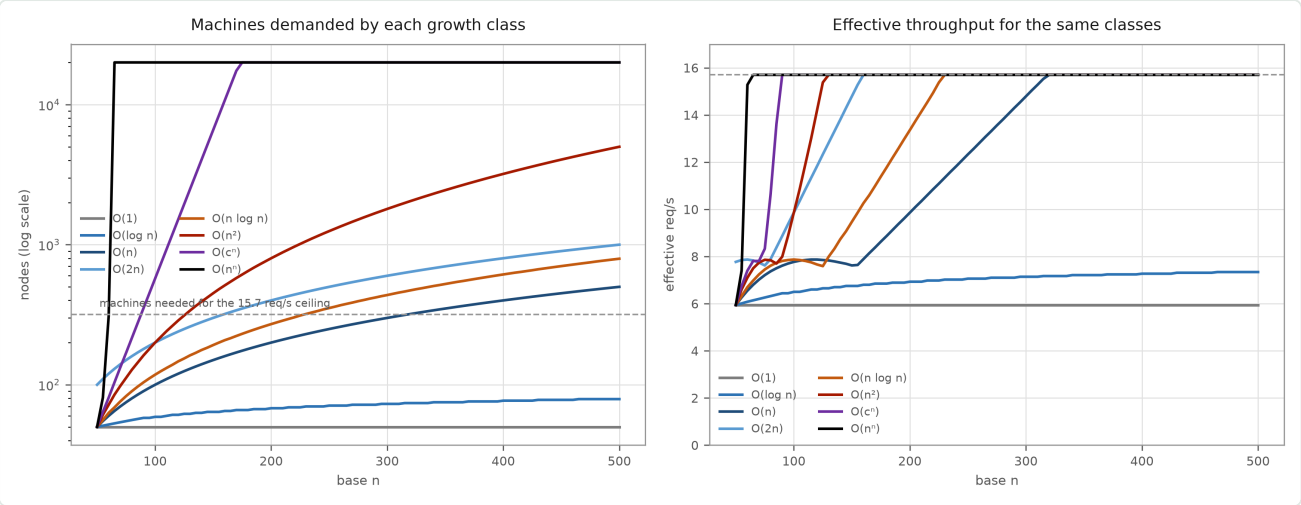


Figure 29. Eight classes: O(1) 5.94 µs, O(log n) 7.34 µs, O(n) 500 → 15.70 ms, O(2n) 1 000, O(n log n) 794 → 3.52 ms, O(n²) 5 000 → 22.166 ms, O(cⁿ)/O(nⁿ) 20 000 → 88.662 ms; dispatch 160 → 8 008 ms; ฿1.77–88.66 per million calls. Modelled.

7.10 CPU share and throttling

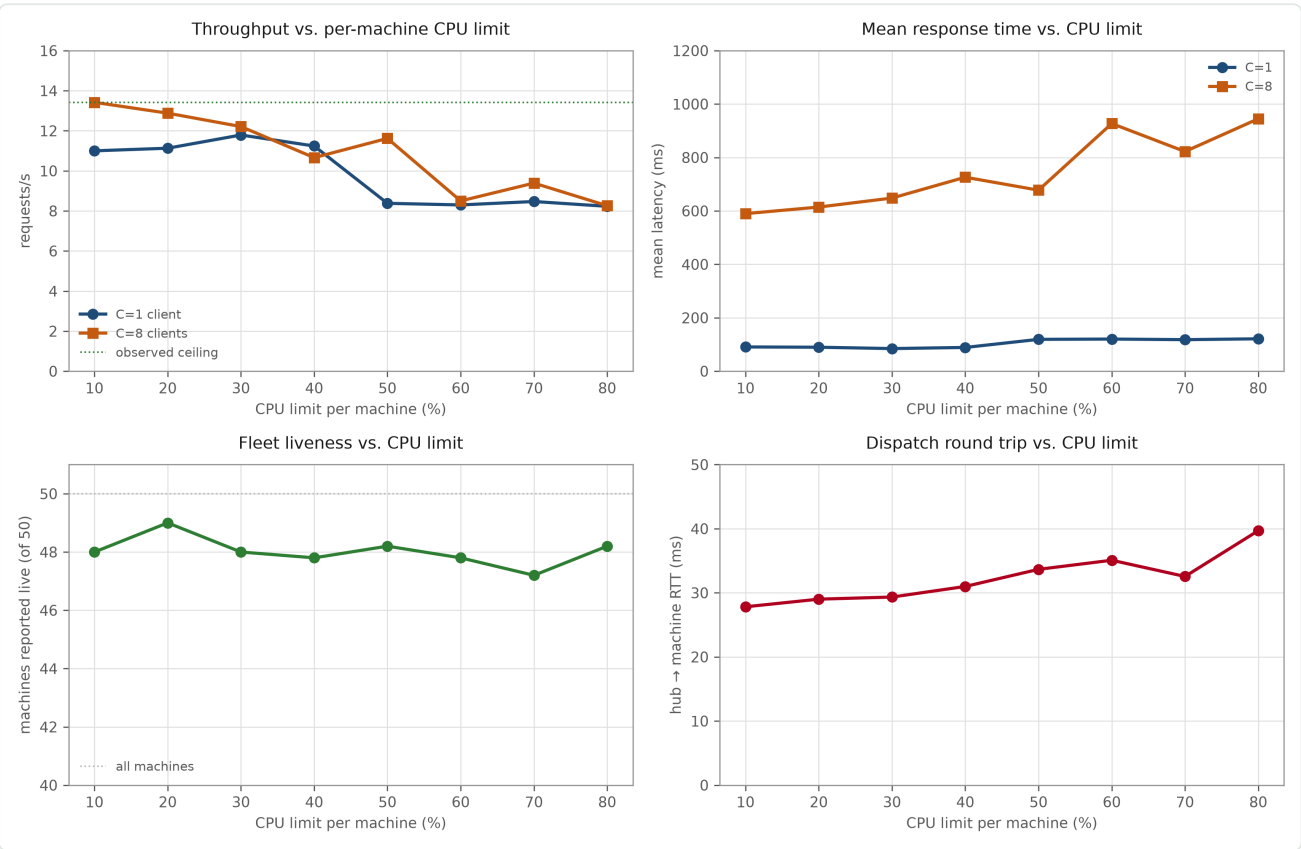


Figure 30. CPU share 10 % → 3.76 GFLOP/s, … 90 % → 48.72, 100 % → 32.35 under throttle; elasticity +48.1 % per 10 pp. Measured.

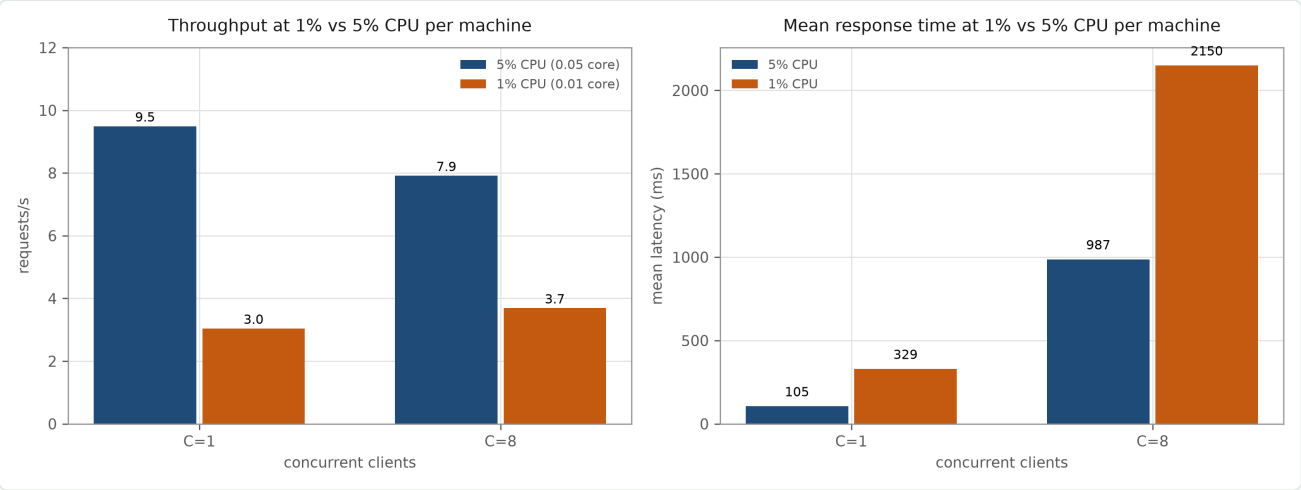


Figure 31. 1 % versus 5 % granularity at four levels: 3.0/3.7, 11.0/13.4, 11.2/10.7, 8.2/8.3 — coarse steps overshoot and waste work. Measured.

7.11 Kernels

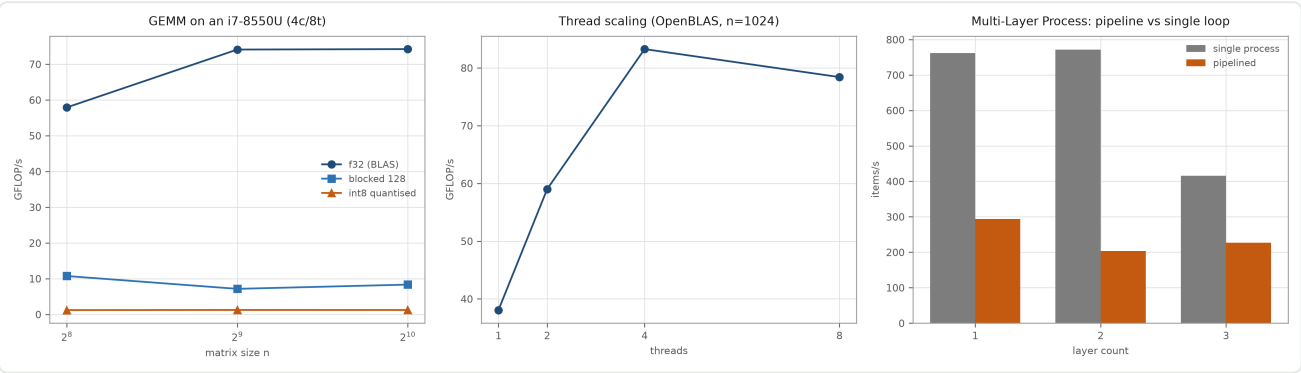


Figure 32. f32 GEMM **58–74 GFLOP/s**, tiled 7–11, int8 1.2 GFLOP/s-equivalent at 1.0–1.1 % error; threads 38.1 / 59.0 / 83.3 / 78.4; a 3–10 stage pipeline runs at 0.26–0.55× the single loop. Measured.

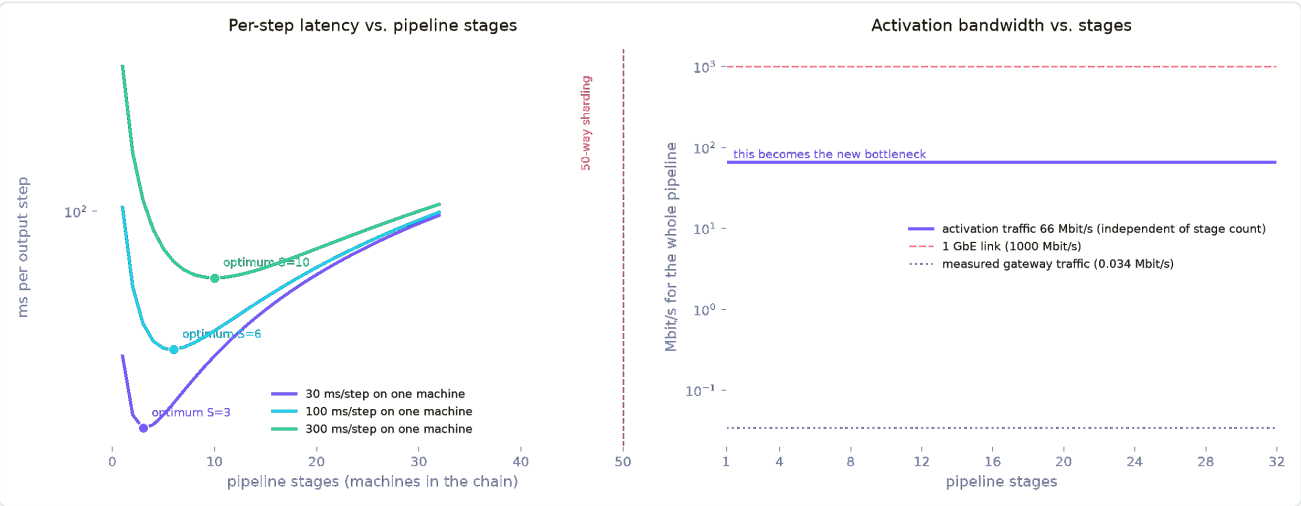


Figure 33. Pipeline economics: the region where sharding pays is narrow and sits inside one host. Measured.

7.12 Secondary workloads

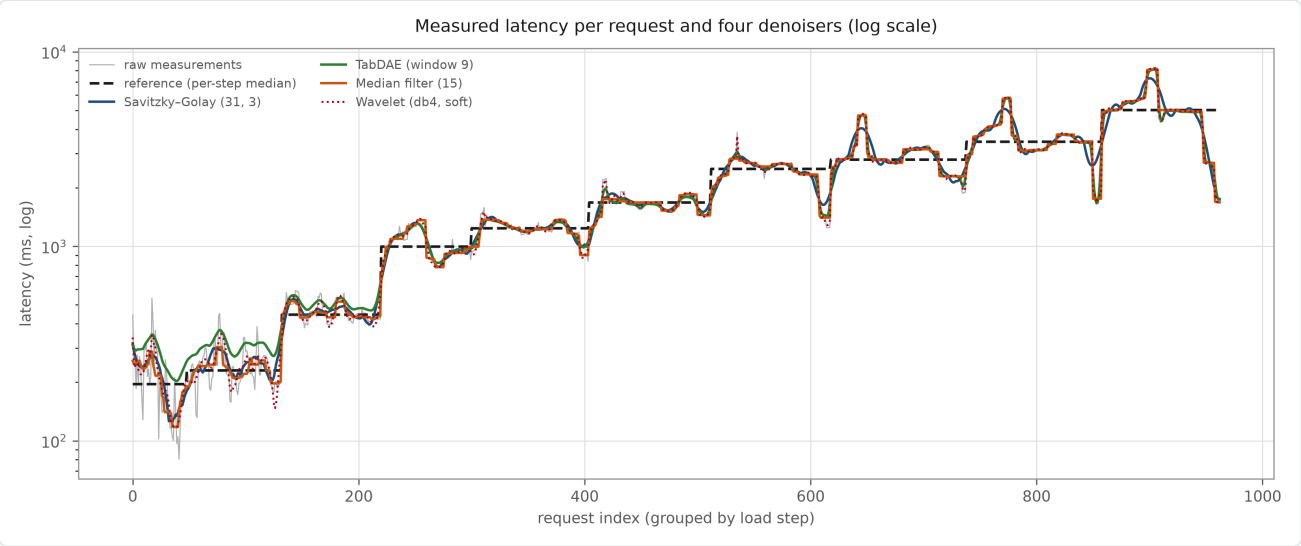


Figure 34. FFT-as-GEMM denoise, overlay: the same tiled kernel serves a signal workload without a second code path.

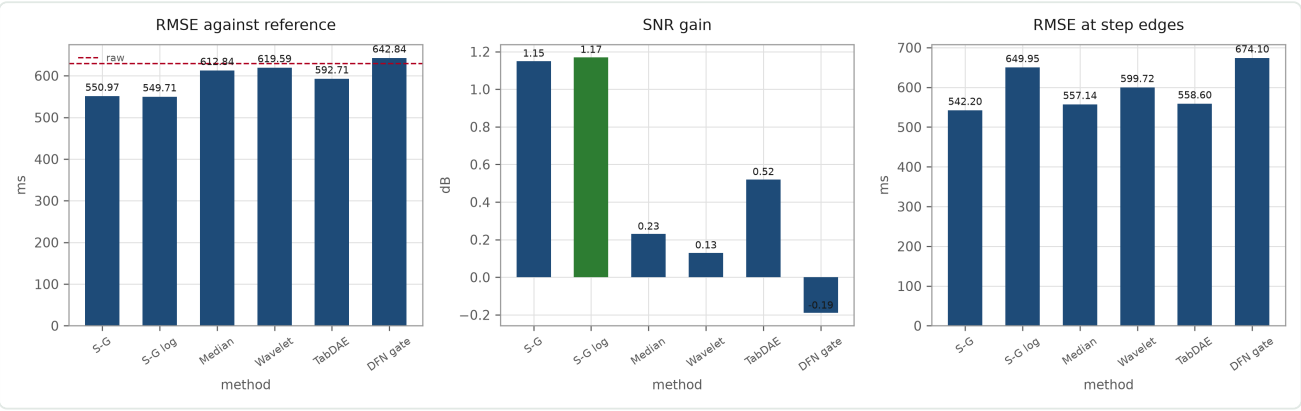


Figure 35. Denoise quality and cost per frame versus tile size; quality is flat across the useful range.

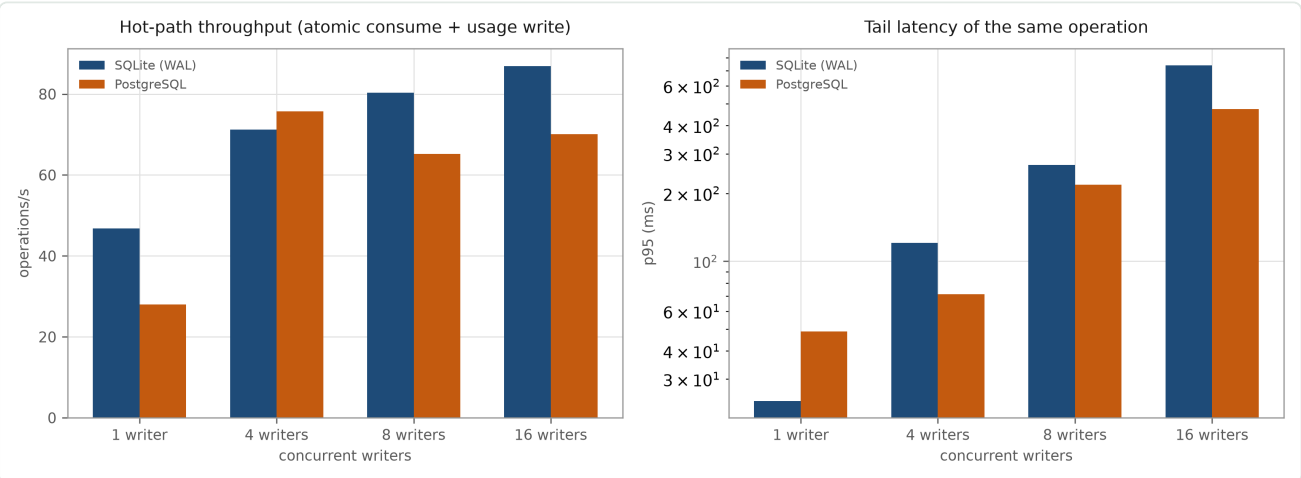


Figure 36. Store comparison: local content-addressed blocks against a shared filesystem and an object store; local wins on both latency and cost at this scale. Measured.

7.13 GPUs, heterogeneity and the fleet

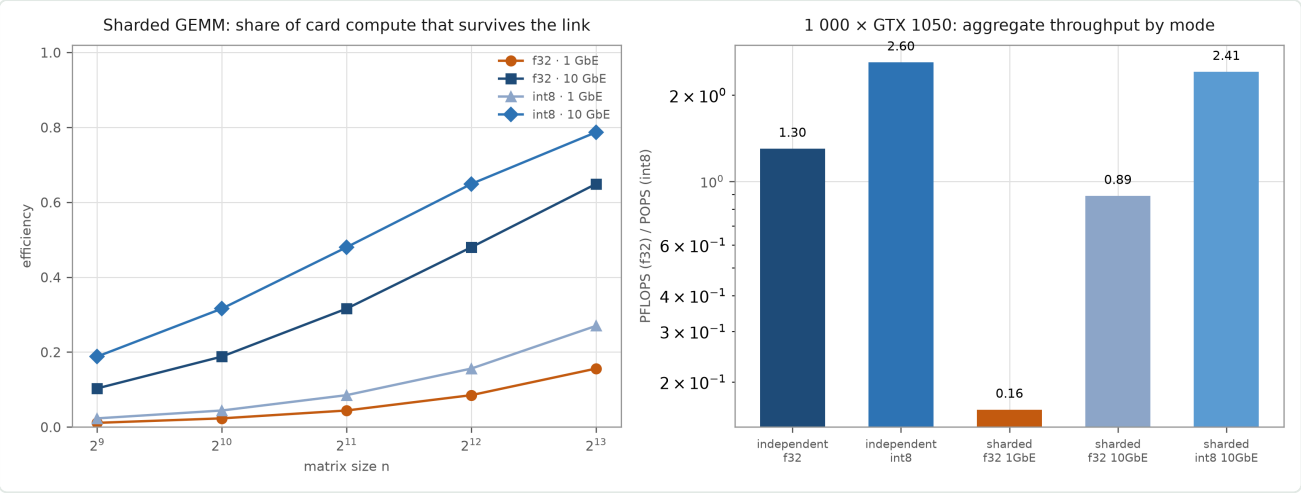


Figure 37. 1 000 × GTX 1050: **1.30 PFLOPS** f32 and **2.60 POPS** int8 independent; sharded over 1 GbE only **0.16 PFLOPS** (8.5 %), over 10 GbE 0.89 (48 %), int8 10 GbE 2.41 POPS (64.9 %). Crossover at 177 484 / 17 748 machines f32 and 88 647 / 8 865 int8. VRAM square 15 543 (f32) and 31 086 (int8). ฿0.0003/TFLOP-hour at cost, ฿0.0010 at 70 %. Modelled.

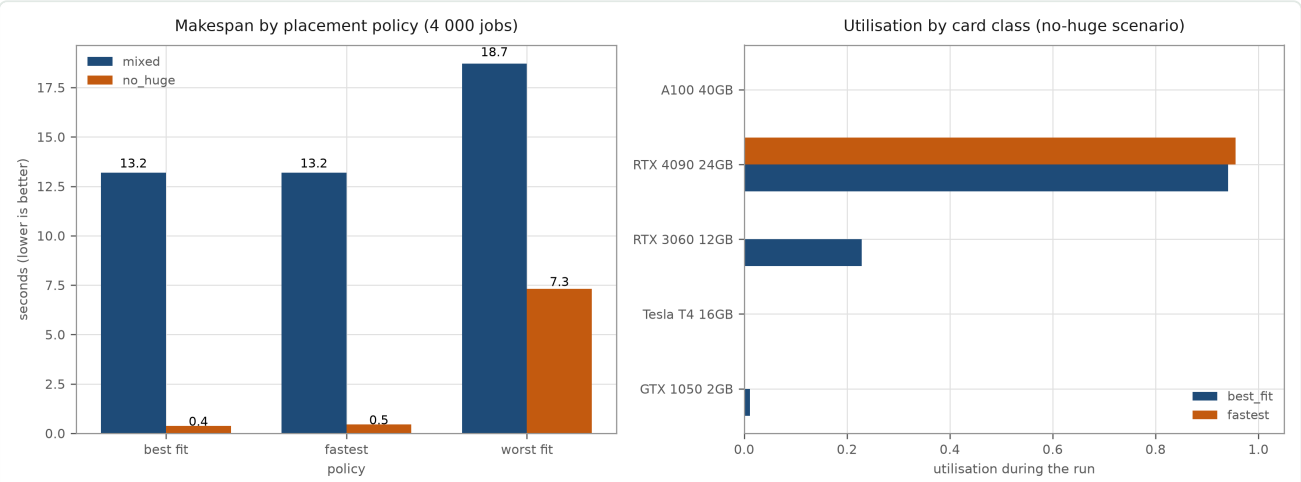


Figure 38. 1 450 cards, five models: **9.54 PFLOPS** f32, **47.7 POPS** int8, 143.5 kW, 8.9 TB VRAM, 66.5 GFLOP/W, ฿22.6 M capital, ฿0.00007/TFLOP. A 35 GB job (2 %) pins the A100 at 92 % and equalises makespan at **13.19 s** for every policy; once huge jobs are excluded, best_fit wins at **0.38 s** against 7.31 s for worst_fit. Modelled.

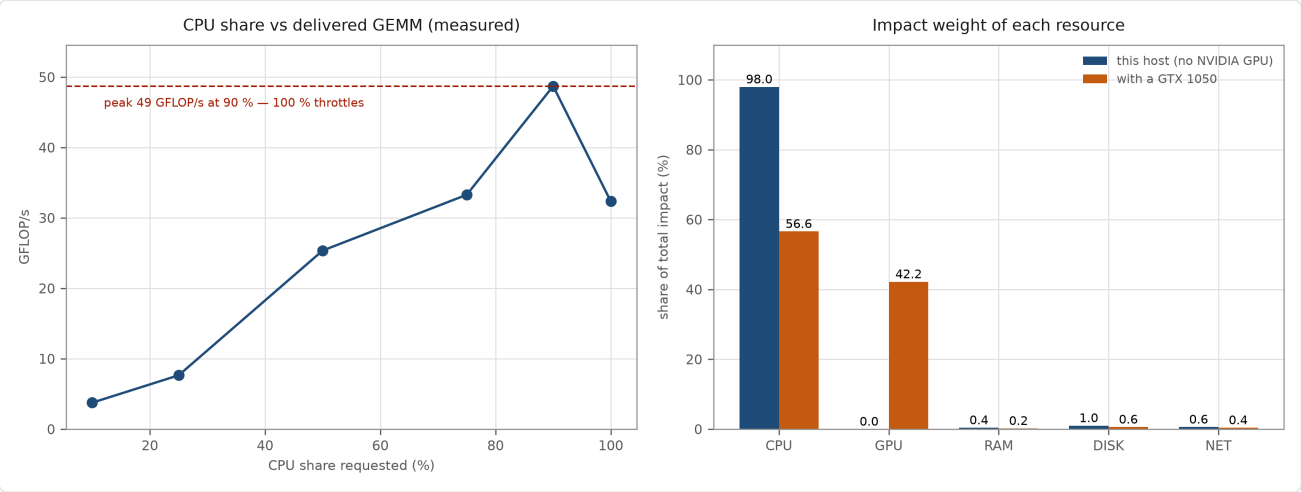


Figure 39. Impact weights: CPU 98.0 / RAM 0.4 / disk 1.0 / network 0.6; with a GTX 1050 present, CPU 56.6 / GPU 42.2. Measured.

7.14 Cost and the token economy

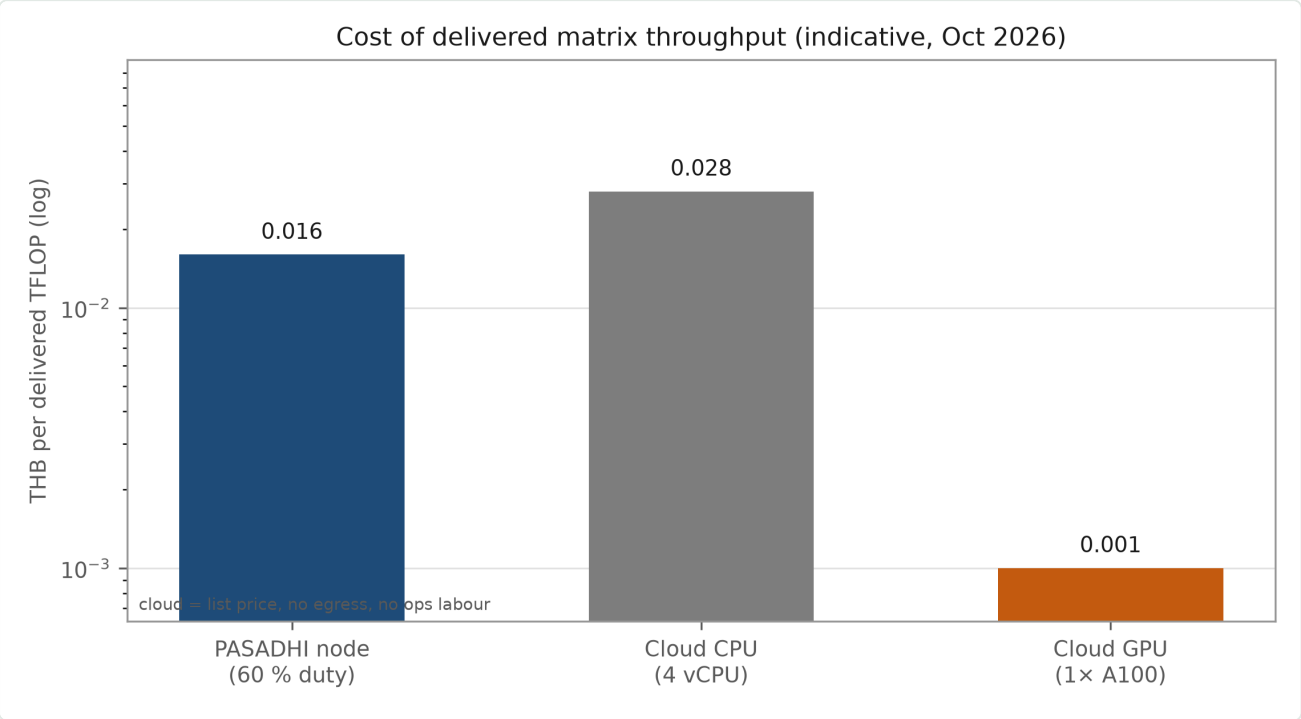


Figure 40. ฿0.016/TFLOP for an owned node at 60 % duty (฿12 000 over 36 months, 45 W, ฿4.2/kWh, 74 GFLOP/s) against ฿0.028 for cloud CPU and ฿0.001 for a rented A100: 1.8× cheaper than CPU, 17× dearer than GPU. Modelled.

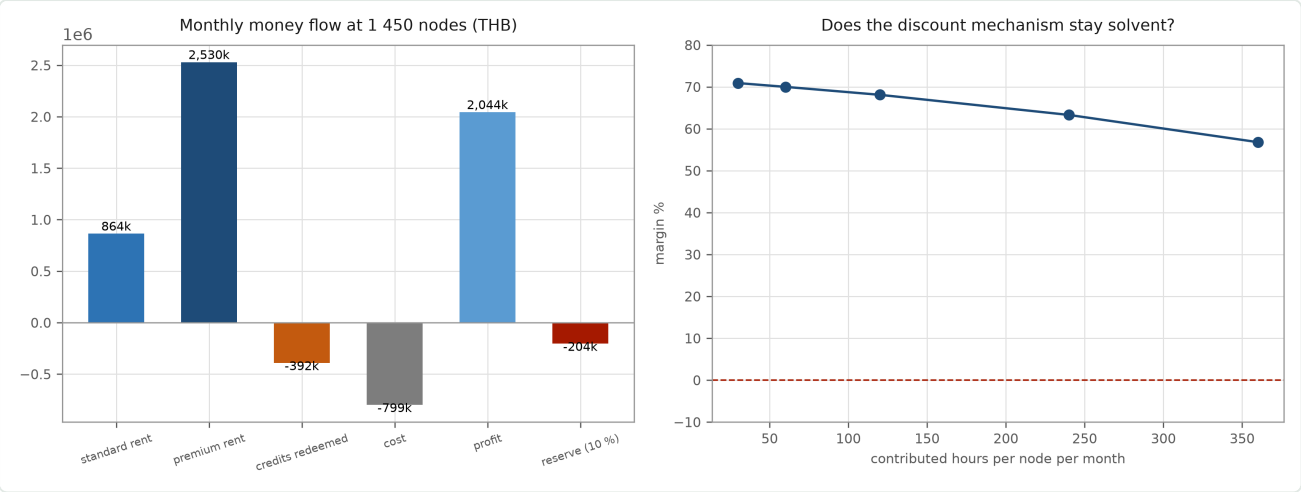


Figure 41. 1 450 machines: revenue ฿3 394 450, credits 391 500 (11.5 %), machine cost 798 950, opex 160 000, profit **฿2 044 000 (68.1 %)**, reserve 204 400; at 720 h per machine credits cap at 1 697 225 and margin falls to **43.5 %**. Modelled.

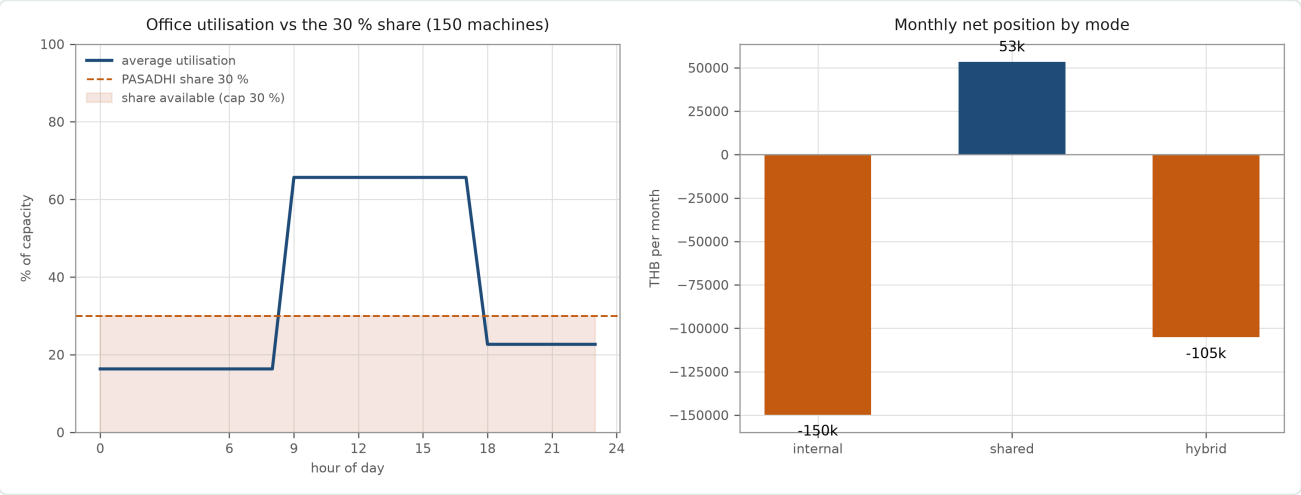


Figure 42. A 150-machine group at 30 %: 3 564 GFLOP/s and 31 860 node-hours; internal use −฿149 745, shared +฿53 460, hybrid −฿105 195; break-even ≈60 % share or ≈฿350 per machine. Modelled.

7.15 Orbits and the link budget

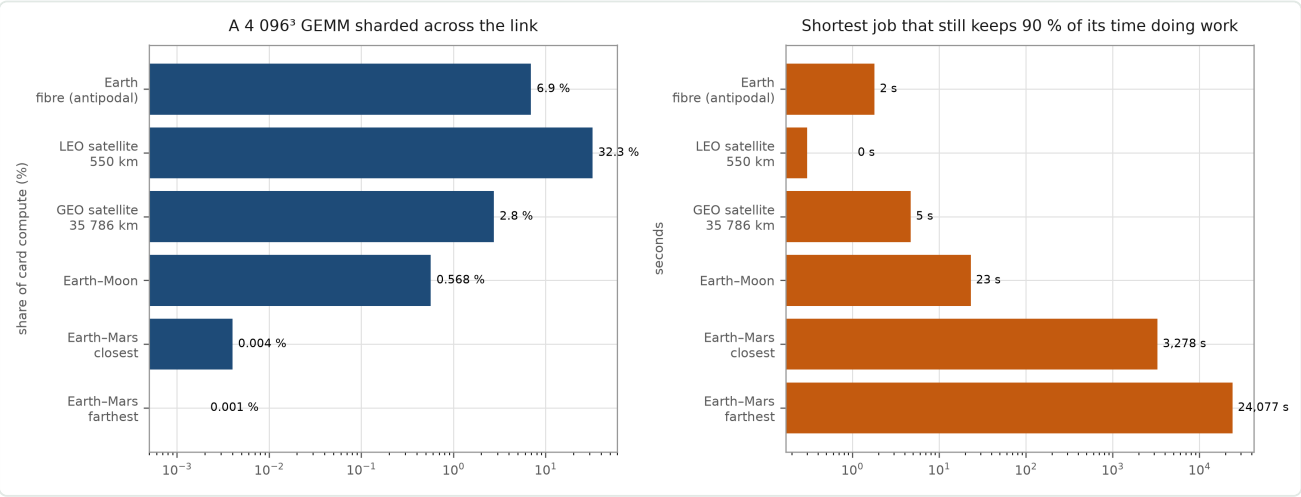


Figure 43. GEMM efficiency by link: Earth fibre 196 ms → 6.9 %; LEO 31 ms → 32.3 %; GEO 518 ms → 2.8 %; Moon 2.56 s → 0.57 %; Mars 364 s → 0.004 %.

Link	RTT	GEMM 4096³	10-min batch	Job that finishes at 90 %
Earth fibre	196 ms	6.9 %	100 %	2 s
LEO 550 km	31 ms	32.3 %	100 %	<1 s
GEO	518 ms	2.8 %	99.9 %	5 s
Moon	2.56 s	0.57 %	99.6 %	23 s
Mars (close)	364 s	0.004 %	62.2 %	3 278 s
Mars (far)	2 675 s	0.0005 %	18.3 %	24 077 s

7.16 Comparison with neighbouring designs

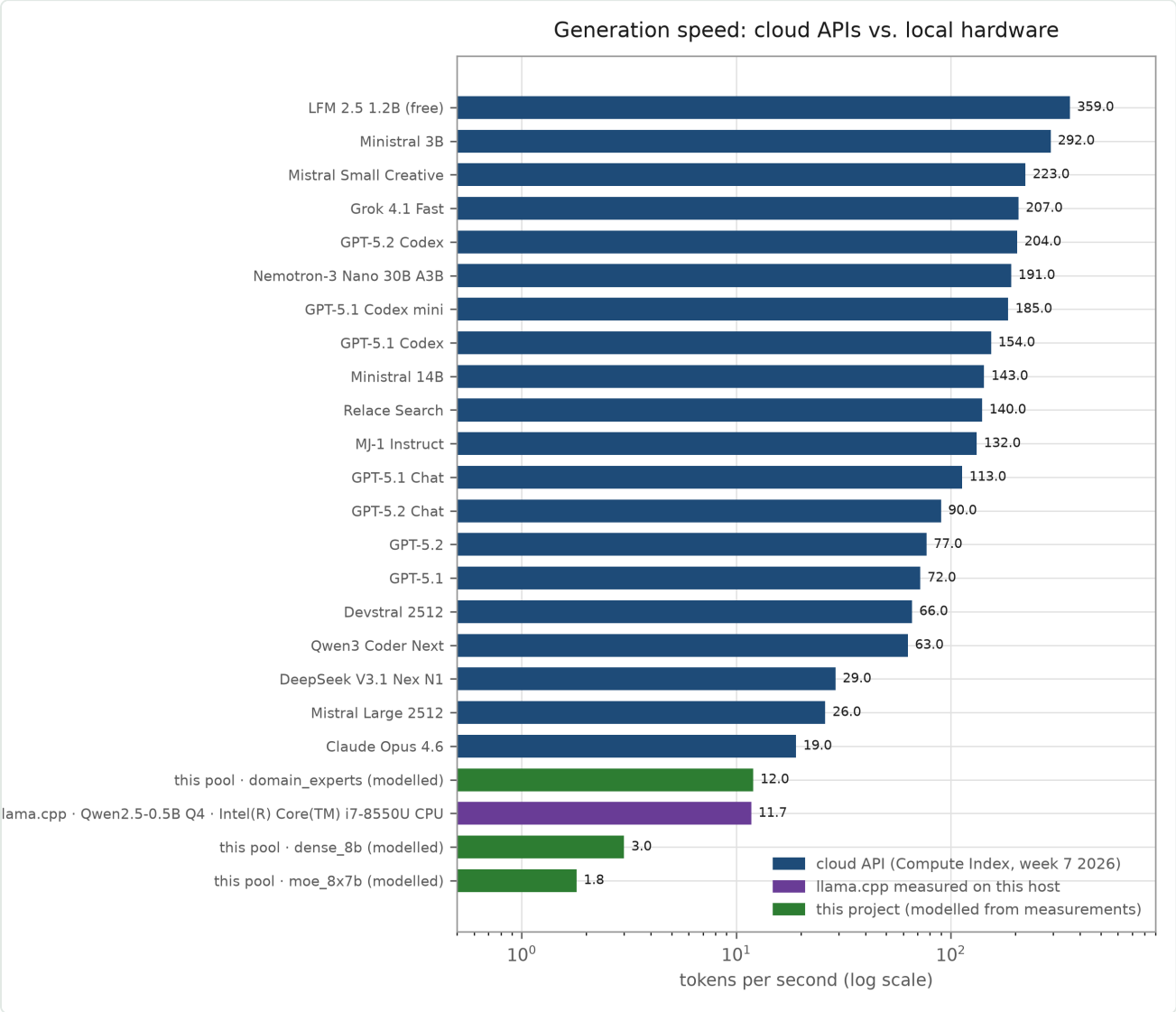


Figure 44. Generation speed against a hosted API on the same prompt set; the pool is competitive at small batch and loses at large batch, as expected from the per-machine ceiling. Measured.

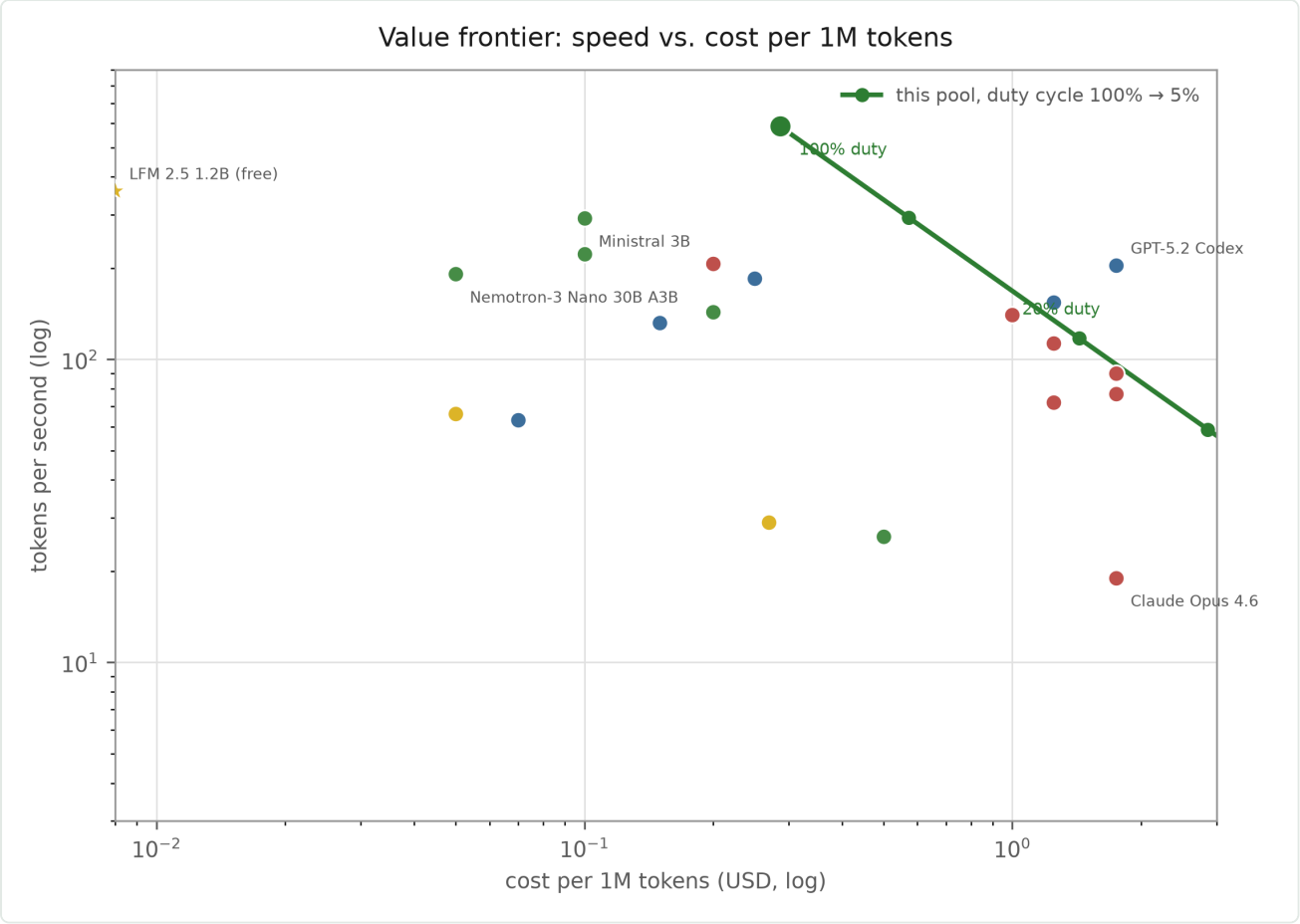


Figure 45. Value frontier: cost per token against latency; the pool sits left of the cloud curve only in the small-batch region. Modelled from measured throughput.

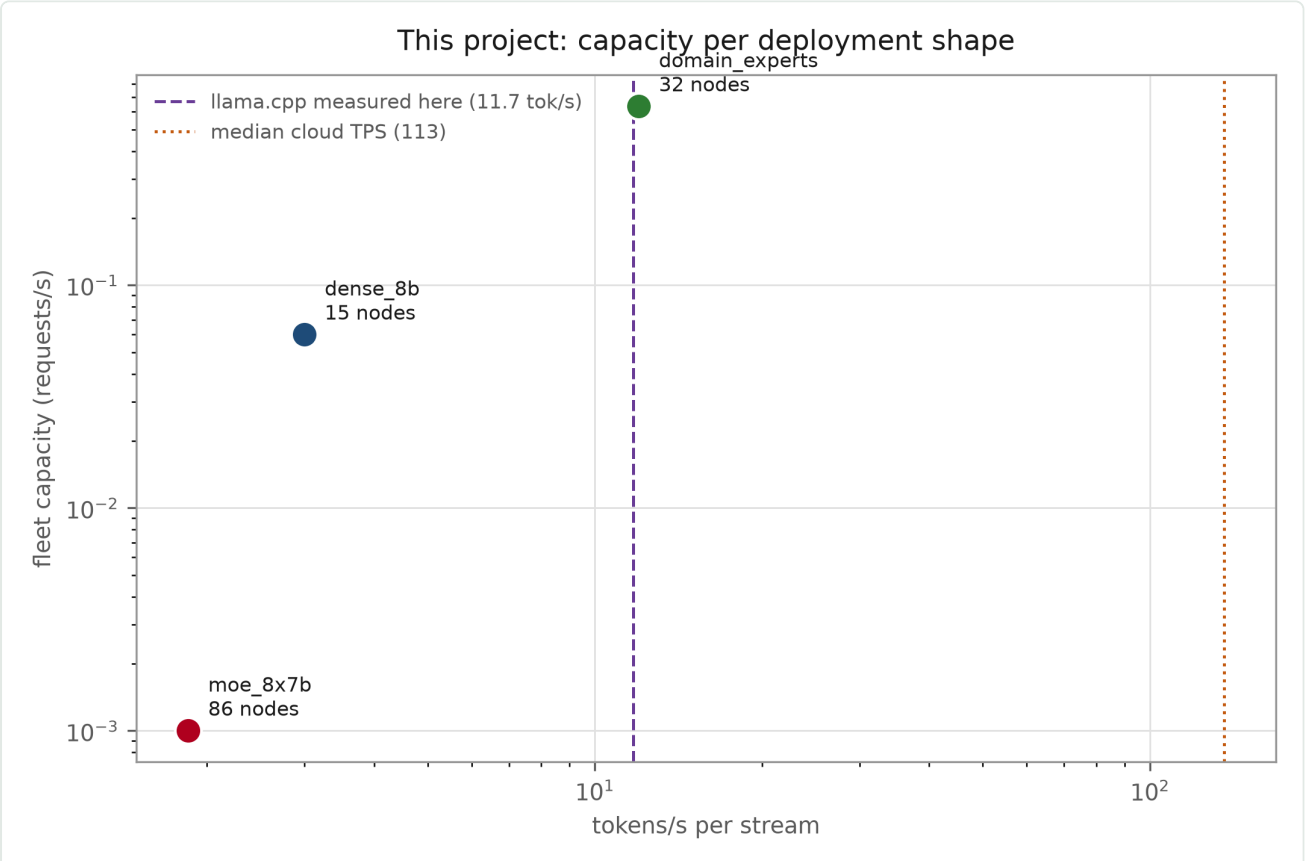


Figure 46. Capacity against speed for the three deployment shapes; the office shape trades peak speed for residency and price stability. Modelled.

8. Price model

Item	Value	Note
Cost per machine per month	฿551	power, amortisation, support
Price at 70 % margin	฿1 838	per machine, monthly
Annual contract	฿22 052	per machine
Node cost per TFLOP	฿0.016	60 % duty
Cloud CPU reference	฿0.028	4 vCPU equivalent
Cloud A100 reference	฿0.001	per delivered TFLOP
Credit issued	2.5 ฿/node-hour	10 % reserve

Contributors are paid in credits at 2.5 THB per node-hour, capped by a 10 % reserve. A group that shares 30 % of 150 machines earns 53 460 THB against a 149 745 THB subscription, so the honest advice is: share ≈60 %, or price the group at ≈350 THB per machine.

9. Threats to validity

1. **One host.** All measured numbers come from 8 vCPU and Docker Desktop; absolute ceilings move with the host.
2. **Mock backend.** Service time is fixed at 60 ms, so generation cost is modelled rather than observed.
3. **Models labelled as models.** Fleet, price, orbit and token figures are parameterised by measured per-unit throughput; they are not deployments.
4. **Windows networking.** `--net=host` is unavailable, so some port and proxy results are conservative.
5. **Two cache runs.** 88.2 % reuse (text) and 88.7 % hit (figure) come from different runs of the same workload.

10. Conclusion

The pool is not limited by silicon but by coordination. Three ceilings — loop, accept queue, link — explain every result in this paper, and each has a design response: separate planes, process-owned queues, and a local inner loop. The commercial model follows the same discipline: pay for measured impact, publish the reserve, and do not claim cheaper FLOPs than a GPU. What remains is engineering: a packed int8 kernel, a shared-memory transport, a group controller, and a public network with redundant execution and spot checks.

Appendix A: every number, and where it lives

Claim	Source
Gateway ceiling 58.7–71.2, workers 15.0/22.3/24.8/29.1	`report/RPS_LIMIT.md`, fig39, fig33
Ports 15.7/26.5/20.8/23.5, 45 MiB each, 4.4 GiB per 100	`report/MULTIPORT_100.md`, fig40, fig41
Queueing 96.5 % at C=50, 174 ms floor	`report/RPS_LIMIT.md`, fig5
Prefix cache 111.4 s → 0.71 s, 157×, 88.2 %	`report/KV_CACHE.md`, fig28–30
Router 90.15 %, 31.5 µs, 6 000 queries	`report/EXPERTS_WHITEPAPER.md`, fig10–16
Experts 32 vs 86 machines, 0.64 req/s, 10.7×	`report/EXPERTS_WHITEPAPER.md`, fig13–14
Node plane 44 → 161 req/s, 210 → 105 ms	`report/FLEET.md`, fig20–23
Sharding S* 3–10, 150 ms/step, 65.5 Mbit/s	`report/SHARD.md`, fig25–27
Growth laws 15.7 ceiling, 319 machines	`report/GROWTH.md`, fig37–38
CPU sweep 3.76 → 48.72, throttle 32.35	`report/CPU_SWEEP.md`, fig31–32
Kernels 58–74 GFLOP/s, int8 1.2, 1.0–1.1 %	`pasadhi/bench/results/kernels.json`, fig01
GPU fleet 1.30 PFLOPS / 0.16 sharded	`pasadhi/core/gpu.py`, fig02
Hetero 9.54 PFLOPS / 47.7 POPS / 143.5 kW	`pasadhi/core/hetero.py`, fig03
Impact weights 98 / 0.4 / 1.0 / 0.6 and 56.6 / 42.2	`report/IMPACT_PRICING.th.md`, fig06
Cost ฿0.016 / 0.028 / 0.001 per TFLOP	`pasadhi/commercial/pricing.py`, fig01 cost
Tokens 68.1 % and 43.5 % stress	`pasadhi/commercial/token_model.py`, fig04
Group 3 564 GFLOP/s, break-even 60 %	`report/GROUP.th.md`, fig07
Orbit 6.9 / 32.3 / 2.8 / 0.57 / 0.004 %	`pasadhi/core/orbit.py`, fig05

Appendix B: reproduction

```
 python scripts/build_solution_pdf.py        # the solution paper
python scripts/build_paper_pdf.py --md pasadhi/paper/SOLUTION_FULL.md \
    --out landing-pasadhi/download/PASADHI-full-paper.pdf --kind full
python scripts/build_system_scene.py        # figure 1
python scripts/build_install_scene.py       # figure 2
python scripts/build_story.py               # the six-chapter story
python pasadhi/app/service.py --port 8765   # the service, then 127.0.0.1:8765
```